

Cómo se hizo



**UNIVERSIDAD
DE GRANADA**

Autor: Lucas Hidalgo Herrera

Grado: Doble Grado en Ingeniería Informática y Matemáticas

Asignatura: Programación Web

Profesor: Serafín Moral García

Fecha: 4 de junio de 2026

Índice General

1 Introducción

A lo largo de este documento, se detallará el proceso de dinamización de la página web de *Azimuth Viajes*. Se tratará en detalle cada paso a seguir; así como los aspectos novedosos que se vayan introduciendo en el proceso de realización.

Antes de continuar debo aclarar que he decidido trabajar con el paradigma de programación dirigida a objetos para *PHP*; de manera que, siempre que se pueda se trabajará con un objeto.

2 Funciones útiles

2.1 PHP

Como cabe esperar, no todas las funciones del back-end que se necesitan pueden estar enmarcadas en una clase que contenga completa funcionalidad; por tanto, he creado un archivo *utils.php* que se encarga de albergar estas funcionalidades; sobre todo, contienen funciones para evitar repetir código o para gestionar el *html* según el tipo de usuario que usa la página web. Estas son:

- *putAvatar*: Es la encargada de escribir en la cabecera el nombre de usuario, su rol y el botón de *log out*; es decir, si es administrador aparece un ordenador y si es usuario aparece el típico avatar indeterminado.
- *putSignUpForm*: Es la encargada de escribir completamente el formulario de inicio de sesión en caso de que ésta no esté iniciada, simplemente es limpieza en el código.
- *putFormSugerencias*: Esta función tiene una funcionalidad análoga a la anterior, su utilidad es limpiar el código del archivo *sugerencias.php* y simplemente escribe el formulario de sugerencias en caso de que la sesión esté iniciada.
- *putHeader*: escribe la cabecera dependiendo del nivel del árbol de archivos que pasemos como argumento; sólo sirve para los niveles 0 y 1 donde 0 es la raíz ¹.
- *putFooter*: escribe el pie de página dependiendo del nivel del árbol de archivos que pasemos como argumento; sólo sirve para los niveles 0 y 1 donde 0 es la raíz ².

3 Conexión a la base de datos

En esta sección, continuamos con la gestión del back-end, es decir, tratando estrictamente con *PHP*; en este caso, hablamos de la conexión a la base de datos. En un

¹Debo decir que la idea de modularizar la cabecera me la indujo una compañera llamada Irina Kuzyshyn Basarab

²Debo decir que la idea de modularizar el pie de página me la indujo una compañera llamada Irina Kuzyshyn Basarab

inicio, traté de realizarlo con una clase que fuera *Conexion* y se encargará de crear y cerrar la conexión a la base de datos mediante un objeto.

Sin embargo, gracias a lo que había en el guión 3 de prácticas y a que, al realizar las demás clases, debía repetir mucho este código, decidí seguir el esquema del guión. En definitiva, he creado una clase abstracta *DataObject* que se encarga de modelar un objeto genérico de la base de datos y del cual heredan todos.

Si nos fijamos no he hablado de las credenciales de la base de datos pero sin ellas no podemos conectarnos. Para ello, he seguido las directrices del guión 3 de prácticas que incita a crear el archivo *config.php* donde se "esconden" las credenciales de la base de datos y todo lo relativo a la conexión. Además, dispone de nombres genéricos para tratar de forma más simple e inequívoca las tablas de la base de datos.

4 El paradigma en JavaScript

Esta sección trata de cómo he gestionado la "comunicación" entre el *front-end* y el *back-end* y es una de las novedades o innovaciones más importantes que he implementado.

Esta se trata del paradigma **AJAJ**, evitando explicaciones es tratar la comunicación como **AJAX** pero con formato *json*. Para ello, me he basado en la gestión de la comunicación asíncrona con *API REST* donde el código *javascript* lanza peticiones *fetch* hacia una *API* para comunicarse con el *back-end*.

4.1 Tratamiento de asincronía

Tal y como he comentado en la sección anterior, estamos ante un paradigma de refresco de página sin recarga y donde *JavaScript* es el protagonista. Al igual que en cualquier comunicación, debemos mandar una pregunta y obtener una respuesta.

En el caso de una comunicación síncrona, aquel que pregunta debe esperar atentamente a que el receptor piense y de una respuesta. Sin embargo, nosotros queremos seguir trabajando mientras el receptor (*back-end*) piensa y formula su respuesta.

Para conseguir eso usaremos las siguientes palabras clave o sentencias:

- *async*: Se usa en la declaración de funciones o métodos de una clase y sirve para determinar que es un método asíncrono. Concretamente, son funciones que devuelven "promesas"; de manera que, si la función devuelve un valor, éste se envuelve en una promesa resuelta y si da error, la promesa será rechazada.

Para entenderlo más fácilmente es como si estoy hablando con una persona, le hago una pregunta y me promete que me va a responder; yo estoy trabajando en otra cosa pero tengo en mente que me tiene que dar una respuesta. Entonces, si esta respuesta es válida y correcta, la procesaré y si no lo es gestionaré el error de comunicación.

- *await*: Sólo puede usarse en entornos gestionados por *async* y se encarga de crear la promesa. Tras esta palabra suele enviarse la pregunta al *back-end* en

forma de datos o suele ponerse una función para recoger los datos de respuesta.

- *fetch*: Sólo voy a explicar lo que he usado; esta sentencia es la encargada de mandar la promesa a la *API* que la procesará ³. La sintaxis básica que yo he usado se compone de los siguientes objetos:
 - *endpoint*: Dirección de la *API* a la que mandar los datos o la petición.
 - *configurationObject*: Es un objeto de tipo diccionario entre llaves que se encarga de definir cómo va a ser la comunicación; algunos de sus atributos son *method* (método *HTTP* para enviar la petición), *ContentType* (tipo de contenido que se envía, habitualmente "application/json") y *body* (cuerpo de la petición con los datos en formato *json*).

Aunque esto puede quedar abstracto, veremos un ejemplo de uso en el alta de usuarios. Por último, y a modo de resumen, podemos ubicar que la metodología para trabajar con *fetch* es la siguiente:

1. Definir el método *async*.
2. Creamos la promesa donde nos comunicamos con la *API* a través de *fetch*.
3. Comprobamos si hay respuesta de la *API* y la conexión ha ido como debería ⁴. En caso contrario gestionamos el error ⁵.
4. "Esperamos" a recibir la respuesta al mensaje completo en formato *json*.
5. Procesamos los datos ya habiendo finalizado la consulta asíncrona.

En definitiva, es un proceso más o menos sencillo igual que otro. En clase vimos cómo hacerlo con *xmlHttpRequest*; sin embargo, buscando información encontré que este método es más novedoso y se está utilizando actualmente en el desarrollo web. Desconozco que el paradigma visto en clase se siga o no utilizando pero, es cierto, que me pareció más complicado. Por eso, elegí este paradigma; además, es una nueva innovación.

4.2 Las API's

Habitualmente, estas *API's* ya están creadas y con plena funcionalidad para ser usadas por los programadores de una forma específica. Sin embargo, yo debo crearlas. Tal y como comenté en un pie de página en la sección ?? están ubicadas en la carpeta 'php/api/' pues no son más que ficheros que se encargan de procesar una entrada en formato *json* y devolver una salida, de nuevo, en formato *json*.

En estas *API's* se debe crear un mensaje *HTTP* al completo con la información a enviar. Las partes que se tratan son:

³En nuestro caso la *API* será normalmente un código *php* que procesa los datos y devuelve errores o éxitos en formato *json*. Habitualmente, estarán ubicadas en la carpeta '/php/api'.

⁴Lo que realmente estamos procesando es la respuesta de la *API* al recibir la cabecera de nuestro mensaje.

⁵Habitualmente lanzando un error.

- *Header*: con la sentencia *Header()* podemos crear y formatear la cabecera del mensaje *HTTP* que vamos a enviar como respuesta.
- *Obtención de datos*: aquí procesamos los datos que debemos recibir; en nuestro caso, usamos la siguiente sentencia:

```
1      $data = json\_decode(file\_get\_contents("php://input"),
      ↪ true);
```

Donde, debido a que *PHP* no puede leer los datos del *body* accediendo a la variable *\$_POST*, accedemos al flujo de entrada en crudo ("php://input"). Además, 'json_decode' se encarga de decodificar el formato y pasarlo a un *array* asociativo de *PHP*.

- *Procesamiento de datos*: en esta parte, procesamos los datos, cada *API* lo hará a su manera.
- *Enviamos respuesta*: fase en la cual debemos responder a la petición. Habitualmente, se utiliza la siguiente sentencia:

```
1      echo json_encode([
2          "success" => true,
3          "message" => "Usuario registrado con éxito"
4      ]);
```

La sentencia *echo* es la forma nativa de escribir datos en el cuerpo de una respuesta *HTTP*; por tanto, estamos enviando la respuesta tras cerrar la ejecución del *script* y codificando en modo *json* el *array* asociativo con las respuestas.

Por tanto, ya hemos visto todo lo necesario para trabajar con el *front-end* y *back-end* de forma asíncrona con *fetch*.

5 Gestión de usuarios

Entramos ya en aspectos importantes, una aplicación web no es nada sin usuarios; por tanto, debemos crear una tabla para ellos en la base de datos. Dicha tabla ('Users') se encarga de modelar a un usuario que dispondrá de los siguientes atributos:

- Nombre
- NickName (PK)
- Email (Unique)
- Password: Este atributo siempre debe aparecer cifrado para que no sea visible para ninguna persona con intenciones maliciosas.

- Admin: Este atributo refleja si el usuario es administrador o no, por defecto, tomará el valor 0 pues el único administrador ya está creado en la base de datos y es el usuario cuyo nickname es 'el_dramas1s'.

Con esta representación podremos gestionar a los usuarios de forma más o menos cómoda. Además, el código está visible en el archivo *user.php*.

5.1 Back-end

Con respecto al cómo se ha modelado para el trato en el servidor (*PHP*), he creado una nueva clase en el archivo *user.php* que hereda de *DataObject* y que modela exactamente las características de los usuarios.

Esta clase, que al principio modelé como una clase instanciable y otra clase distinta que lo manejaba, se encarga ahora de hacer ambas cosas usando métodos de clase y métodos de objeto.

Referente a los métodos de objetos, son los encargados de gestionar las operaciones de la base de datos, así como de realizar algunas verificaciones de los objetos en cuestión. Estos métodos crean usuarios en la base de datos, los eliminan, los buscan y verifican que la contraseña y el *nickName* pasados como argumentos estén asociados.

Para el caso de los métodos de objeto, tenemos un método *getter* para obtener los datos, un método de validación de consistencia de un usuario y un método para comprobar si es admin.

5.1.1 Alta de usuarios dinámica

Comienza lo interesante, debemos ahora gestionar la alta de nuevos usuarios en la página web; por tanto, deberemos tener en cuenta que debemos comprobar el formulario de alta de usuario. Esta tarea no solo es del front-end sino que también del back-end pues nos dará mayor protección frente a fallos del usuario o del front-end.

Por tanto, he creado un archivo *forms.php* que se encarga de manejar los formularios. Este archivo sigue la misma lógica de árbol de herencia que los *DataObjects* pues disponemos de un manejador que se encarga de modelar el comportamiento básico como una clase abstracta que aparece a continuación:

```

1  <?php
2  abstract class FormHandler
3  {
4      protected $errors = [];
5
6      /**
7       * Se encarga de validar el formulario y procesarlo
8       *
9       * @param array $data
10      * @return bool True si el formulario se ha procesado correctamente,
11      * ↪ false en caso contrario (o si hay errores de validación)
12      */

```

```

12 public function handle(array $data): bool
13 {
14     $this->validate($data);
15
16     if (empty($this->errors)) {
17         return $this->process($data);
18     }
19
20     return false;
21 }
22
23 /**
24  * Devuelve los errores que tenga el formulario
25  *
26  * @return array Un array con los errores encontrados, vacío si no hay
27  *   ↳ errores
28  */
29 public function getErrors(): array
30 {
31     return $this->errors;
32 }
33
34 /**
35  * Valida el formulario
36  *
37  * @param array $data
38  * @return array Un array con los errores encontrados, vacío si no hay
39  *   ↳ errores
40  */
41 abstract public function validate(array $data);
42
43 /**
44  * Procesa la información del usuario
45  *
46  * @param array $data
47  * @return bool|User|string True si el proceso se ha realizado
48  *   ↳ correctamente, un mensaje de error en caso contrario, o un objeto
49  *   ↳ User si se ha creado uno nuevo
50  */
51 abstract public function process(array $data);
52
53 /**
54  * Añade un error a la lista de errores
55  *
56  * @param string $field
57  * @param string $message
58  * @return void
59  */
60 protected function addError(string $field, string $message)
61 {
62     $this->errors[$field] = $message;
63 }
64 }
65 ?>

```


Donde aparecen las cuatro funciones básicas de un formulario:

- **Validar entradas:** La función *validate* se encarga de validar las entradas del objeto, cada formulario tendrá sus validaciones dependiendo del objeto que se esté tratando.
- **Procesar entradas:** La función *process* se encarga de procesar el formulario dándole la funcionalidad que merece. Dependiendo del formulario que se trate realizará unas operaciones u otras.
- **Gestionar posibles errores:** Esta funcionalidad la realizan dos funciones, *addError* y *getErrors*, junto con el dato miembro de la clase que es el array de errores.

Por tanto, cada vez que queramos gestionar un formulario, lo haremos a través de la función *handle* que engloba las dos primeras. Y si devuelve *false* procesaremos los errores con *getErrors*.

Nos centramos ya en el formulario de alta de usuarios, cuyo manejador es *FormSignUp* que se encarga de obtener los datos, validarlos y, si cumple las características básicas para poder crear un usuario, se crea en la base de datos. La única característica que queda fuera de las básicas es una condición de política; esta es: "un usuario no puede ser menor de edad"⁶ ⁷.

El código de este manejador aparece en el *listing ??*.

```
1  <?php
2
3  class FormSignUp extends FormHandler
4  {
5      public function validate(array $data)
6      {
7          $this->errors = [];
8
9          if (empty($data['nombre'])) {
10             $this->addError('nombre', 'El nombre es obligatorio');
11          } elseif (strlen($data['nombre']) > 30) {
12             $this->addError('nombre', 'El nombre no puede tener más de 30
              ↳ caracteres');
13          } elseif (!preg_match('/^[A-Za-zÁÉÍÓÚáéíóúÑñÜü ]+$/',
              ↳ $data['nombre'])) {
14             $this->addError('nombre', 'El nombre solo puede contener letras
              ↳ y espacios');
15          }
16
17          if (empty($data['nickName'])) {
18             $this->addError('nickName', 'El nickname es obligatorio');
19          } elseif (!preg_match('/^[a-zA-Z0-9_]+$/', $data['nickName'])) {
```

⁶El motivo de esto es evitar pagos de usuarios menores de edad, en caso de falsificarlo es problema del usuario.

⁷Si nos fijamos, no había ningún atributo de mayoría de edad en la base de datos para el usuario; el motivo es que si está en la base de datos es porque se mayor de edad.

```

20         $this->addError('nickName', 'Solo letras, números y _');
21     } elseif (strlen($data['nickName']) < 3 || strlen($data['nickName'])
    ↪ > 30) {
22         $this->addError('nickName', 'Debe tener entre 3 y 30
    ↪ caracteres');
23     }
24
25     if (empty($data['email'])) {
26         $this->addError('email', 'El email es obligatorio');
27     } elseif (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
28         $this->addError('email', 'Email no válido');
29     }
30
31     if (empty($data['password'])) {
32         $this->addError('password', 'La contraseña es obligatoria');
33     } elseif (strlen($data['password']) < 8 || strlen($data['password'])
    ↪ > 15) {
34         $this->addError('password', 'Debe tener entre 8 y 15
    ↪ caracteres');
35     } elseif (!preg_match(
36         '/(?!.*[a-z])(?!.*[A-Z])(?!.*\d)(?!.*[!@#$%^&*~.]).{8,15}/',
    ↪ $data['password'])) {
37         $this->addError('password', 'Debe contener mayúscula, minúscula,
    ↪ número y símbolo');
38     }
39
40     if (!isset($data['aceptar'])) {
41         $this->addError('aceptar', 'Debes aceptar el aviso legal');
42     }
43
44     if (!isset($data['adulthood'])) {
45         $this->addError('adulthood', 'Debes indicar si eres mayor de
    ↪ edad');
46     }
47 }
48
49 public function process(array $data)
50 {
51     $esAdulto = isset($data['adulthood']) && (int)$data['adulthood'] === 1;
52
53     if (!$esAdulto) {
54         $this->addError('adulthood', 'Debes ser mayor de edad');
55         return false;
56     }
57
58     $user = new User([
59         'nickName' => $data['nickName'],
60         'nombre' => $data['nombre'],
61         'email' => $data['email'],
62         'password' => $data['password'],
63         'admin' => 0
64     ]);
65
66     $errores = $user->validateUser();
67

```

```

68         if (!empty($errores)) {
69             foreach ($errores as $error) {
70                 $this->addError('general', $error);
71             }
72             return false;
73         }
74
75         if (User::exists($data['nickName'], $data['email'])) {
76             $this->addError('general', 'Nickname o email ya en uso');
77             return false;
78         }
79
80         if (!User::create($user->getDatos())) {
81             $this->addError('general', 'Error al crear usuario');
82             return false;
83         }
84
85         return true;
86     }
87 }
88 ?>

```

Ahora, pensando en como funciona nuestra comunicación con el *front-end* a través de *fetch*, debemos crear una *API*; tal y como comenté en la sección ??, se encuentra en el archivo *altausuarios.php* dentro de la carpeta */php/api*. Este archivo contiene el siguiente código:

```

1  <?php
2
3  require_once __DIR__ . '/../model/forms.php';
4
5  header('Content-Type: application/json');
6
7  $data = json_decode(file_get_contents("php://input"), true);
8
9  if (!$data) {
10     echo json_encode([
11         "success" => false,
12         "message" => "No se han recibido datos"
13     ]);
14     exit;
15 }
16
17 $form = new FormSignUp();
18 $status = $form->handle($data);
19
20 if (!$status) {
21     echo json_encode([
22         "success" => false,
23         "errors" => $form->getErrors()
24     ]);
25     exit;

```

```

26   }
27
28   echo json_encode([
29       "success" => true,
30       "message" => "¡Usuario registrado correctamente!"
31   ]);
32
33   exit;
34   ?>

```

En él aparece la gestión, básicamente procesa entradas desde *javascript* hacia el *back-end* creando un manejador de formularios y aplicando el método *handle*. De esta forma, el archivo *altausuarios.html* no cambiaría ⁸. Además, con la gestión realizada con *javascript* que realizaré después no es necesario utilizar el archivo *validacion.html* que teníamos en la segunda práctica. Por tanto, esta *API* procesa los datos recibidos en formato *json* y devuelve una respuesta en cuyo *body* está en formato *json*.

5.2 Front-end

Para entender esto, lo primero que debemos entender es **¿qué debe hacer el *front-end* en esta parte?**. Concretamente, debe comprobar que los datos del formulario de alta de usuarios sean correctos. Para ello, vamos a usar la clase *formularioAlta* que hereda de *formularioBase* y de la cual heredarán todos los formularios.

Comencemos explicando un poco el código de *formularioBase*; la idea es clara, emular el modelo realizado en *php* sobre los formularios. Por tanto, esta clase se tomará como clase abstracta que define el comportamiento completo de un formulario cualquiera. Además, como no tenemos que conectar con la base de datos al estar en *front-end* es más fácil de definir el comportamiento. El código aparece a continuación:

```

1  export class formularioBase {
2      /**
3       * Construye un objeto formulario tomándolo del archivo en el que se
4       * ↪ ejecuta
5       *
6       * @param {string} idFormulario identificador del formulario
7       */
8      constructor(idFormulario) {
9          this.form = document.getElementById(idFormulario);
10
11          this.form.addEventListener('submit', (e) => this.enviar(e));
12
13          this.realTimeValidation();
14      }

```

⁸La modularización del pie de página provoca que sea *.php*

```

15  /**
16   * Construye un objeto que representa las entradas del formulario
17   *
18   * @returns Objeto cuyas entradas son los datos del formulario
19   */
20  obtenerDatos() {
21      const datos = new FormData(this.form);
22
23      return Object.fromEntries(datos.entries());
24  }
25
26  /**
27   * Valida los datos, se implementa en las hijas
28   *
29   * @param {Object} datos datos a validar
30   */
31  validar(datos) {
32      const reglas = this.reglas();
33      const errors = {};
34
35      for (const campo in reglas) {
36          const validador = reglas[campo];
37
38          const error = validador(datos[campo])
39
40          if (error) {
41              errors[campo] = error;
42          }
43      }
44
45      return errors;
46  }
47
48
49  /**
50   * Envia los datos al servidor para procesarlos
51   *
52   * @param {Event} event
53   */
54  async enviar(event) {
55      event.preventDefault();
56
57      const reglas = this.reglas();
58
59      for (const campo in reglas) {
60          this.limpiarError(campo);
61      }
62
63      const datos = this.obtenerDatos();
64      const errors = this.validar(datos);
65
66      if (Object.keys(errors).length > 0) {
67          for (const field in errors) {
68              if (field == 'password') {
69                  this.form.elements['password'].value = "";

```

```

70         }
71         this.mostrarError(field, errors[field]);
72     }
73 }
74 else {
75     //Enviamos los datos
76     try {
77         //Hacemos el fetch asíncrono
78         const response = await fetch(this.endpoint, {
79             method: 'POST',
80             headers: { 'Content-Type': 'application/json' },
81             body: JSON.stringify(datos)
82         });
83
84         if (!response.ok) {
85             throw new Error(`Error en la red: ${response.status}`);
86         }
87
88         //Si hay respuesta la procesamos pasandola a json
89         const json = await response.json();
90
91         //Procesamos la respuesta
92         if (json.success) {
93             this.onSuccess(json);
94         } else {
95             //Si hay errores los procesamos
96             if (json.errors) {
97                 for (const field in json.errors) {
98                     this.mostrarError(field, json.errors[field]);
99                 }
100             //Si no hay errores procesamos el mensaje
101             } else if (json.message) {
102                 alert(json.message);
103             }
104         }
105     } catch (error) {
106         console.error("Hubo un problema con la petición fetch:",
107             ↪ error);
108         alert("No se pudo conectar con el servidor. Inténtalo de
109             ↪ nuevo más tarde.");
110     }
111 }
112
113 /**
114  * Crea un mensaje de error en el formulario
115  *
116  * @param {string} fieldName nombre del campo del formulario
117  * @param {string} message mensaje de error
118  */
119 mostrarError(fieldName, message) {
120
121     if (fieldName === 'general') {
122         alert(message);

```

```

123         location.reload();
124         return;
125     }
126
127     let field = this.form.elements[fieldName];
128     //Si es un radio button, nos quedamos con el primero
129     if (field instanceof RadioNodeList || field.length > 0) {
130         field = field[0];
131     }
132     if (field) {
133
134         let container = field.closest(".field")
135         if (container) {
136             container.classList.add("error")
137
138             let pError = container.querySelector(".errorMessage");
139
140             if (!pError) {
141
142                 pError = document.createElement("div");
143                 pError.classList.add("errorMessage");
144
145                 container.appendChild(pError);
146             }
147             pError.textContent = message;
148         }
149     }
150 }
151
152 /**
153  * Limpia el error que habia en el campo con nombre fieldName
154  *
155  * @param {string} fieldName nombre del campo
156  */
157 limpiarError(fieldName) {
158     let field = this.form.elements[fieldName];
159
160     //Si son los radio buttons pues cojemos el primero solo
161     if (field instanceof RadioNodeList || field.length > 0) {
162         field = field[0];
163     }
164
165     if (field) {
166         let container = field.closest(".field");
167
168         if (container) {
169
170             container.classList.remove("error");
171             let pError = container.querySelector(".errorMessage");
172
173             if (pError) {
174                 pError.remove()
175             }
176         }
177     }

```

```

178     }
179
180     /**
181     * Devuelve las reglas de cada campo de un formulario
182     *
183     * @returns diccionario con las reglas de cada campo
184     */
185     reglas() {
186         return {}
187     }
188
189     /**
190     * Procesa el json en caso de exito
191     *
192     * @param {json} json Mensaje de validacion
193     * @returns Depende de la clase
194     */
195     onSuccess(json) {
196         return;
197     }
198
199     realTimeValidation() {
200
201         const eventos = ['input', 'change'];
202
203         eventos.forEach(tipoEvento => {
204             //Imponemos un eventListener en los eventos recogidos
205             this.form.addEventListener(tipoEvento, (e) => {
206                 const field = e.target.name;
207                 const reglas = this.reglas();
208
209                 if (reglas[field]) {
210                     this.limpiarError(field);
211
212                     const datos = this.obtenerDatos();
213                     const error = reglas[field](datos[field]);
214
215                     if (error) {
216                         this.mostrarError(field, error);
217                     }
218                 }
219             });
220         });
221     }
222 }
223
224 }
225

```

Este comportamiento se basa en un *eventListener* que se activa al darle al botón de enviar y funciona de la siguiente forma:

1. Anulamos el comportamiento por defecto del navegador ante este evento con

preventDefault para evitar que recargue la página, queremos controlar todo lo que pasa en el navegador.

2. Obtenemos las reglas del formulario que hemos instanciado. Las reglas es una función que dependerá del formulario en cuestión y las definimos como una función encargada de devolver un diccionario de funciones que asocia una función a cada campo; se puede entender como una receta de recetas. En estas reglas podemos definir el comportamiento que ve el usuario, como por ejemplo, hacer "click" en un enlace obligatorio.
3. Limpiamos los errores que hubiera en un principio en caso de que no fuera el primer intento del formulario para poder poner otros en caso de corregir los que había. La función de limpieza es clara; puesto que lo que hago para mostrar los errores es crear un contenedor con el error dentro de un contenedor que contiene los campos, bastará con eliminarlo completamente para que, en caso de necesitarlo, crear otro nuevo y no modificar demasiado el espaciado entre campos ⁹.
4. Obtenemos los datos del formulario con *FormData*¹⁰ y los convertimos en un array asociativo con la función *Object.fromEntries()*.
5. Validamos los datos del formulario aplicando las reglas a cada uno de ellos.
6. Si hay errores los mostramos creando un contenedor de error dentro del contenedor del campo. En el caso de tener un *radioButton* lo que hacemos es tomar el primero de ellos como referente para la construcción del error.
7. Si no hay errores realizamos la comunicación con *fetch* usando el *endpoint* del formulario. Cabe destacar que cada uno dispone de su *endpoint* o *API* del *back-end* para procesar sus envíos.

Además, si el manejador de formulario del *back-end* detecta algún error como ser mayor de edad, lo devuelve en el *body* del mensaje *HTTP* de respuesta y al procesarla en el *front-end* se muestra el error en el campo correspondiente.

Es importante recalcar que la función de envío se trata de forma asíncrona como se explicó en la sección ???. Además, hay una cosa que no he comentado y que es un añadido, pues con lo que he explicado todo funciona correctamente. Si miramos el código, en el constructor aparece la función *realTimeValidation* que, como dice su nombre, se encarga de procesar la validación en tiempo real del formulario.

Esta función es sencilla, simplemente hay que añadir un *eventListener* al formulario por cada evento del tipo que queramos aplicando las reglas y limpiando los errores si todo está correcto. En caso de no estar correcto, simplemente muestra el error.

Pasando ahora al comportamiento concreto del alta de usuarios, éste se define en

⁹Esto hace referencia a que, una vez probé a crear los contenedores de error y simplemente cambiar el interior a "" si no existía ese error. Esto solo consiguió que aumentara el gap entre campos de forma notable.

¹⁰Es una interfaz nativa que permite construir fácilmente conjuntos de datos en pares clave-valor para enviarlos al servidor

el archivo *formularioAlta.js* donde está la clase *formularioAlta* y que hereda de *formularioBase*. Por tanto, solo debemos asignar el *endpoint* que en este caso es *../php/api/altausuarios.php*, definir el booleano de *avisoLeido* para forzar al usuario a leer el aviso legal simulando términos y condiciones y crear las reglas del formulario.

Antes de explicar las reglas, me gustaría explicar una experiencia que tuve con *avisoLeido*. En un inicio, esta bandera se gestionaba en la función de reglas, es decir, se ponía a falso y, con un *eventListener* en el enlace se cambiaba a *true* cuando se hacía "click". Sin embargo, tras un largo tiempo pensando por qué no funcionaba, me di cuenta de que no era una gestión correcta pues, cada vez que se ejecutaban las reglas, notaba como que el enlace no era "clickado" lo cual era cierto. Por tanto, decidí pasarlo a dato miembro del objeto, lo cual permitía guardar el cambio de form a permanente.

Las reglas se recogen en el código que aparece a continuación referente a la clase hija de *formularioBase* y que se corresponde con el alta de usuarios. Creo que es bueno verlo una vez para entender el funcionamiento, sólo devuelve funciones para poder validar los campos.

```
1  reglas(){
2      const nameRegex = /^[A-Za-zÁÉÍÓÚáéíóúÑñÜü ]+$/;
3      const nickNameRegex = /^[a-zA-Z0-9_]+$/;
4      const emailRegex =
5      ↪ /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
6      const passwordRegex =
7      ↪ /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[$@#!%*?&.])(8,15)/;
8
9      // Variable para controlar si se ha leído el aviso legal
10
11      ↪ document.getElementById("enlaceLegal").addEventListener('click', ()=>{
12          this.avisoLeido = true;
13      });
14
15      return {
16          nombre : (value) => {
17              if(!value || value.trim() === "") return "Nombre vacío";
18              if(value.length > 30) return "Máximo 30 caracteres";
19              if(!nameRegex.test(value)) return "Solo letras y espacios";
20              return null;
21          },
22
23          nickName : (value) => {
24              if(!value || value.trim() === "") return "NickName vacío";
25              if(value.length < 3 || value.length > 30) return "Entre 3 y
26              ↪ 30 caracteres";
27              if(!nickNameRegex.test(value)) return "Solo letras, números
28              ↪ y _";
29              return null;
30          },
31
32          email : (value) => {
33              if(!value || value.trim() === "") return "Email vacío";
```

```

29         if(value.length > 100) return "Máximo 100 caracteres";
30         if(!emailRegex.test(value)) return "Email no válido";
31         return null;
32     },
33
34     password : (value) => {
35         if(!value || value.trim() === "") return "Contraseña vacía";
36         if(value.length < 8 || value.length > 15) return "Debe tener
37         ↪ entre 8 y 15 caracteres";
38         if(!passwordRegex.test(value)) return "Debe contener
39         ↪ mayúscula, minúscula, número y símbolo";
40         return null;
41     },
42
43     aceptar : (value) => {
44         if(!value || value.trim() === "") return "Debes aceptar el
45         ↪ aviso legal";
46         if(!this.avisoLeido) return "Debes leer el aviso legal";
47         return null;
48     },
49
50     adultez : (value) => {
51         if(!value || value.trim() === "") return "Debes confirmar
52         ↪ que eres mayor de edad";
53         return null;
54     }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }

```

Sin embargo, debemos añadir a *main.js* una nueva sentencia para crear e instanciar un manejador de este tipo:

```

1  if(document.getElementById("formularioAlta")){
2      new formularioAlta();
3  }

```

Una vez hecho esto, añadimos el script a la página de alta de usuarios.

6 Gestión de sesiones

Pasamos a algo importante, cada usuario debe tener una sesión y para ello, debemos usar el formulario de inicio de sesión. Una vez el usuario ya tiene un perfil creado, puede loguearse y ver aquellas funcionalidades más importantes.

6.1 Back-end

En el servidor seguiremos la misma lógica que con la creación de usuarios; debemos gestionar el formulario de inicio de sesión. Utilizaremos una *API* que procese los datos recibidos por *fetch* y actuaremos arreglo a esos usando un manejador del formulario. En este caso es muy sencillo, simplemente comprueba que las credenciales coinciden en la base de datos aplicando el *hash* correspondiente y buscando si el usuario existe y, si todo es correcto almacena información del usuario en el array `$_SESSION`. El código de esta parte es el siguiente:

```
1  <?php
2
3  class FormLogIn extends FormHandler
4  {
5      public function validate(array $data)
6      {
7          $this->errors = [];
8
9          if (empty($data['nickName'])) {
10             $this->addError('nickName', 'El nickname es obligatorio');
11         }
12
13         if (empty($data['password'])) {
14             $this->addError('password', 'La contraseña es obligatoria');
15         }
16     }
17
18     public function process(array $data)
19     {
20         if (
21             !User::exists($data['nickName'], '') ||
22             !User::verifyPassword($data['nickName'], $data['password'])
23         ) {
24             $this->addError('general', 'Nickname o contraseña incorrectos');
25             return false;
26         }
27
28         $_SESSION['nickName'] = $data['nickName'];
29         $_SESSION['loggedIn'] = true;
30         $_SESSION['admin'] =
31             ↪ User::getUserByNickname($data['nickName'])->isAdmin();
32
33         return true;
34     }
35 }
36
37 //API de inicio de sesión
38 <?php
39 //Necesitamos iniciar la sesión dentro de la api porque
40 //el formulario trabaja directamente con $_SESSION
41 session_start();
42
43 require_once __DIR__.'../../model/forms.php';
```

```

44 require_once __DIR__.'../utils.php';
45
46 header('Content-Type: application/json');
47
48 $data = json_decode(file_get_contents("php://input"),true);
49
50 if(!$data){
51     echo json_encode([
52         "success" => false,
53         "message" => "No se han recibido datos"
54     ]);
55     exit;
56 }
57
58 $form = new FormLogIn();
59 $status = $form->handle($data);
60
61 if(!$status){
62     echo json_encode([
63         "success" => false,
64         "errors" => $form->getErrors()
65     ]);
66     exit;
67 }
68
69 echo json_encode([
70     "success" => true,
71     "message" => "¡Usuario loggeado correctamente!"
72 ]);
73
74 exit;
75 ?>

```

Una vez ya hemos hablado del formulario y de la *API*, que son bastante sencillos, debemos de hablar sobre cómo utiliza esto el servidor para mostrar unas cosas u otras. Trataremos el código de *index.php* y con eso entenderemos el funcionamiento de otros archivos que usen estos aspectos.

```

1  <?php
2  //Iniciamos la sesión
3  session_start();
4  require_once './php/utils.php';
5  ?>
6
7  <!DOCTYPE html>
8  <html lang="es">
9
10 <head>
11     <meta charset="UTF-8">
12     <meta name="viewport" content="width=device-width, initial-scale=1.0">
13     <link rel="stylesheet" href="./css/style.css">
14     <link rel="stylesheet" href="./css/form.css">

```

```

15     <link rel="stylesheet" href="./css/index.css">
16     <title>Azimut Viajes | Exclusividad en cada sitio</title>
17 </head>
18
19 <body>
20     <?php putHeader(); ?>
21     <main id="mainIndex">
22         <section id="introSection">
23             <article>
24                 <h2>Azimut Viajes</h2>
25                 <div class="carrusel" id="carrusel">
26                 </div>
27                 <p>Bienvenido a Azimut Viajes, tu agencia de viajes de
                ↳ confianza para descubrir los mejores destinos del mundo.
                ↳ Aquí encontrarás las mejores ofertas para tu próximo
                ↳ viaje. Seremos tu <strong>rumbo</strong> y tu
                ↳ <strong>dirección</strong> en cada aventura.</p>
28             </article>
29         </section>
30         <?php if (isset($_SESSION['loggedIn']) && $_SESSION['loggedIn'] ==
31         ↳ true): ?>
32             <section id="searchSection">
33                 <form id="formularioBusqueda" method="get"
34                 ↳ action="./html/viajes_buscados.php" autocomplete="off"
35                 ↳ target="_self">
36
37                 <fieldset>
38                     <datalist id="continentes">
39                         <option value="Europa">Europa</option>
40                         <option value="Asia">Asia</option>
41                         <option value="America">América</option>
42                         <option value="Africa">África</option>
43                         <option value="Oceania">Oceanía</option>
44                     </datalist>
45                     <input type="text" id="continent" name="continent"
46                     ↳ placeholder="Continente" list="continentes">
47
48                     <datalist id="países"></datalist>
49                     <input type="text" id="country" name="country"
50                     ↳ placeholder="País" list="países">
51
52                     <datalist id="places"></datalist>
53                     <input type="text" id="place" name="place"
54                     ↳ placeholder="Lugar" list="places">
55
56                     <label for="fecha_salida">From:</label>
57                     <input type="date" id="departureDate"
58                     ↳ name="departureDate"
59                     ↳ min="<?php echo date('Y-m-d'); ?>">
60
61                     <label for="fecha_regreso">To:</label>
62                     <input type="date" id="returnDate" name="returnDate"
63                     ↳ min="<?php echo date('Y-m-d'); ?>">
64
65                     <button type="submit"></button>
61     </fieldset>
62
63     </form>
64
65     </section>
66     <?php endif ?>
67 </main>
68 <?php putFooter(); ?>
69 <script type="module" src="./js/main.js"></script>
70 </body>
71
72 </html>
73
74 <?php
75 function putHeader($level=0){
76     echo "<header>
77
78         <h1 id=\"nombreDecor\">Azimut Viajes</h1>";
79
80     if($level == 0){
81         echo "<img id=\"logoHeader\" src=\"../imagenes/logoAzimut.png\"
82             ↳ alt=\"Logotipo Azimut\">";
83     } else if($level == 1){
84         echo "<img id=\"logoHeader\" src=\"../imagenes/logoAzimut.png\"
85             ↳ alt=\"Logotipo Azimut\">";
86     }
87
88     //Si no se ha loggeado, pongo el formulario
89     if (!isset($_SESSION['loggedIn']) || $_SESSION['loggedIn'] === false) {
90         if($level == 0){
91             putsignUpForm();
92         } else if($level == 1){
93             echo '<a href="../index.php" id="Session" style="grid-area:
94                 ↳ signUpForm; justify-self: center;">Inicia sesión</a>';
95         }
96     }
97     } else { //Si se ha loggeado ponemos el avatar con la inicial del
98     ↳ nickname y el boton de cierre de sesión
99     putAvatar($_SESSION['admin'], $_SESSION['nickName'], $level);
100 }
101
102 if($level == 0){
103     echo "<nav id=\"menuHeader\">
104         <ul>
105             <li><a href=\"../index.php\">Inicio</a></li>
106             <li><a href=\"../html/viajes.php?page=1\">Viajes</a></li>
107             <li><a href=\"../html/viajes_grupo.php\">Viajes en
108                 grupo</a></li>
109             <li><a href=\"../html/ofertas.php\">Ofertas</a></li>
110             <li><a href=\"../html/sobre_agencia.php\">Sobre nuestra
111                 agencia</a></li>
112             <li><a href=\"../html/sugerencias.php\">Sugerencias</a></li>
113         </ul>

```

```

110         </nav>";
111     } else if($level == 1){
112         echo "<nav id=\"menuHeader\">
113             <ul>
114                 <li><a href=\"../index.php\">Inicio</a></li>
115                 <li><a href=\"../html/viajes.php?page=1\">Viajes</a></li>
116                 <li><a href=\"../html/viajes_grupo.php\">Viajes en
117                     grupo</a></li>
118                 <li><a href=\"../html/ofertas.php\">Ofertas</a></li>
119                 <li><a href=\"../html/sobre_agencia.php\">Sobre nuestra
120                     agencia</a></li>
121                 <li><a href=\"../html/sugerencias.php\">Sugerencias</a></li>
122             </ul>
123         </nav>";
124     }
125     echo "</header>";
126 }
127 ?>

```

Como vemos, siempre que queramos hacer uso de la sesión debemos iniciarla con la sentencia `session_start()`. Realmente, no siempre la iniciamos, el funcionamiento que he encontrado es el siguiente:

- **PHPSESSID**: Cuando iniciamos una sesión por primera vez, el servidor envía una *cookie* que contiene el id de sesión y crea un archivo con la información del array `$_SESSION`.
- **`session_start()`**: Cuando usamos esta función en *php* lo que hacemos es comprobar que el navegador ha mandado ese id de sesión al servidor, en caso de que lo tengamos ¹¹. El servidor reinicia la conversación y podemos usar la información almacenada. Además, se encarga de aumentar el tiempo de vida de la sesión.

Los aspectos que tratamos en algunos archivos son los siguientes:

- **Perfil del usuario**: Con esto me refiero a la imagen de perfil y el *nickName*. Cuando ya se ha iniciado sesión, se recarga la página y el formulario de inicio de sesión se cambia por el nombre de usuario junto a su rol y el botón de cierre de sesión.
- **Sugerencias**: Para la empresa *Azimut Viajes*, una sugerencia sin autor no es nada; por tanto, debes ser usuario para poder dejar una sugerencia.
- **Buscador**: Considero que es buena idea que, si el usuario no se ha registrado, no pueda ser capaz de realizar búsquedas. En caso de que quiera ver los viajes tendrá la posibilidad de verlos todos en la sección de viajes de la página web.

En adición, aunque lo explique en la sección ??, cabe recalcar que para gestionar las funcionalidades del administrador se hace uso del campo *admin* del array asociativo

¹¹Si no lo tenemos hacemos el punto anterior.

de las sesiones. Dependiendo de su valor, rederizamos unas cosas u otras ¹².

Por último, para cerrar la sesión, cuando se pulsa el botón de cerrar sesión se manda ejecutar el siguiente código donde se destruye la sesión y se borran todos los datos de la misma:

```
1  <?php
2
3  session_start();
4
5  // Vaciamos la sesion para eliminar cualquier dato del usuario
6
7  $_SESSION = [];
8
9  // Destruimos la sesion para asegurarnos de que se cierre la sesion
10 session_destroy();
11
12 header('Location: ../index.php');
13
14 //Detenemos la ejecucion del script, php no lo hace tras una redireccion
15 exit();
16 ?>
```

6.2 Front-end

Con lo explicado en la sección ?? hay poco que explicar. Solo debemos saber que para gestionar esto debemos recaer en una nueva clase hija de *formularioBase*; ésta se llamara *formularioSesion* y se encargará de tomar el *endpoint* correcto y de gestionar las reglas de validación oportunas. El código es el siguiente:

```
1  //////////////////////////////////
2  // FORMULARIO INICIO SESION
3  //////////////////////////////////
4
5  //Importamos la clase genérica
6
7  import { formularioBase } from "../formularioBase.js";
8
9  export class formularioSesion extends formularioBase{
10
11      /**
12       * Constructor del manejador del formulario de inicio de sesion
13       */
14      constructor(){
15          super("signUpForm");
16          this.endpoint = "../php/api/inicioSesion.php"
```

¹²Considero que esa es la mejor opcion, pues mandarlas como invisibles permite que el navegador conozca los aspectos que trata un administrador y hacerlos visibles es tan fácil como modificar el código en el navegador.

```

17     }
18
19     /**
20      * Devuelve las reglas asociadas al formulario de inicio de sesion
21      * @returns Array asociativo de funciones
22      */
23     reglas(){
24         const nickNameRegex = /^[a-zA-Z0-9_]+$/;
25         const passwordRegex =
26             ↪ /(?(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[$@#!%*?&.] ).{8,15})/;
27         return {
28             nickName: (value) => {
29                 if(!value || value.trim() === "") return "NickName vacío";
30                 if(value.length < 3 || value.length > 30) return "Entre 3 y
31                 ↪ 30 caracteres";
32                 if(!nickNameRegex.test(value)) return "Solo letras, números
33                 ↪ y -";
34                 return null;
35             },
36
37             password: (value) => {
38                 if(!value || value.trim() === "") return "Contraseña vacía";
39                 if(value.length < 8 || value.length > 15) return "Debe tener
40                 ↪ entre 8 y 15 caracteres";
41                 if(!passwordRegex.test(value)) return "Debe contener
42                 ↪ mayúscula, minúscula, número y símbolo";
43                 return null;
44             }
45         }
46     }
47
48     /**
49      * Respuesta del servidor cuando sí se loguea el usuario
50      *
51      * @param {json} json
52      */
53     onSuccess(json){
54         location.reload();
55     }
56 }

```

Como podemos ver, el diseño que he realizado para procesar elementos en el *front-end* es bastante cómodo y limpio. ¡Se me olvidaba! Debemos añadir el *script* al archivo *index.php* y, también, añadir una nueva sentencia al archivo *main.js*; aparece justo aquí debajo esa sentencia ¹³:

¹³Esta sentencia se ejecuta igual para todos los manejadores de formularios de *javascript*, con los usuarios ocurría igual.

```

1  if(document.getElementById("signUpForm")){
2      new formularioSesion();
3  }

```

7 El administrador

Esta sección no dispone de trabajo en el *front-end* puesto que este usuario no se crea como los demás, sino que ya está creado en la base de datos. Actualmente hay dos, uno para desarrollo y pruebas, y otro que permanecerá en la base de datos.

Además, la forma de trabajar con ellos que he decidido no involucra trato con *javascript*. Principalmente, esto es porque he decidido que, aquel código que no sea necesario que viaje por la red, no viajará. Para ello, he utilizado las sesiones, donde el atributo `$_SESSION['admin']` guarda la información del usuario registrado referente a si es administrado o no ¹⁴.

Siguiendo este principio, tenemos ya la justificación de por qué hay ciertas sentencias condicionantes, como la que aparece justo abajo, para decidir qué partes son de administradores y qué partes no.

```

1  <?php if (isset($_SESSION['admin']) && $_SESSION['admin'] === true): ?>

```

```

1  <?php
2      //Mostrar los botones
3      if (isset($_SESSION['admin']) && $_SESSION['admin'] && $general) {
4          $html .= "<button class=\"deleteButton\"
5                  data-continent=\"\" .
6                      ↪ htmlspecialchars($this->datos['continent']) .
7                      ↪ \"\"
8                  data-country=\"\" .
9                      ↪ htmlspecialchars($this->datos['country']) . \"\"
10                 data-place=\"\" .
11                 ↪ htmlspecialchars($this->datos['place']) . \"\">
12                 <img src=\"../imagenes/deleteButton.png\"
13                 ↪ alt=\"Eliminar\">
14                 </button>";
15      }
16  ?>

```

¹⁴Esta información se gestiona con el formulario de inicio de sesión que forma parte del *front-end* y está directamente ligada con la *API* de inicio de sesión que devuelve si es o no administrador

Vuelvo a recalcar, creo que es un fallo gestionar la visibilidad de estos aspectos desde el *front-end* pues el código es visible desde el inspector y, un usuario promedio, no debería conocer, siquiera, las funcionalidades referentes a administración. Por tanto, no mandarlo es la mejor opción.

He hablado de como tratar con los administradores, pero no he hablado sobre por qué un usuario no puede crearse como administrador. El motivo es sencillo: porque en el alta de usuarios no se trabaja ese valor y se interpreta que todo usuario que use ese formulario deberá ser usuario sin privilegios. Además, dentro de la base de datos, la columna de administrador toma el valor 0 por defecto, ayudando a que la gestión sea más segura.

8 CRUD de Viajes

Vamos con la parte más importante de la práctica, el trato de viajes. Comenzaremos hablando de la estructura interna que soporta todo el trabajo y de las clases que he implementado para trabajar con ella y pasaremos a hablar de cada una de las partes del *CRUD*.

Antes de nada, cabe destacar que todas estas funcionalidades sólo serán visibles si el usuario que usa la aplicación es un administrador; en caso contrario, el código relativo a estas funcionalidades no saldrá del servidor.

8.1 Estructura de viajes

Me refiero a la base de datos, es decir, a las tablas que guardan la información relacionada con los viajes. He creado dos tablas y son las siguientes:

- **infoTrips**: se encarga de modelar los destinos, es decir, contiene el continente, el país, el lugar, el precio, la ruta de la imagen, las descripciones y los alojamientos.
- **Trips**: modela cuándo se va a producir un viaje; por tanto, guarda parejas destino-fechas. Donde destino no es más que el continente, el país y el lugar y las fechas son la fecha de salida y la de llegada.

El motivo por el cual decidí seguir esta implementación, que creo que está normalizada hasta tercera forma normal, es sencillo. De esta forma evito guardar información duplicada y puedo tener tantos viajes a un destino como quiera, siempre y cuando no se repitan ambas fechas.

Con esta estructura he podido modelar el *CRUD* de los viajes de *Azimut Viajes*.

8.2 Clases necesarias

Las clases que he implementado son dos, una por cada tabla y, realmente no modelan los objetos como tal. La clase *infoTrips* sí que modela completamente la tabla asociada. Sin embargo, *Trip* no cumple esta premisa; podría parecer un error y, la

verdad, desconozco si lo es, pero me ha permitido trabajar con la union de ambas tablas que es la entidad "Viaje" que realmente estamos tratando.

Por tanto, modela el *JOIN* de ambas tablas. La mayoría de funciones de las que disponen son básicas pero trataremos algunas de ellas:

- *infoTrips::delete*: esta función es un arma de doble filo, disponemos de claves externas de la clave primaria de *infoTrips(continent, country, place)* de la tabla *Trips* hacia *infoTrips*; por tanto, realizar el borrado tal y como aparece es un error. No obstante, como no se usa así y esto se controla en *Trips*, podemos trabajar así.

```
1  <?php
2      /**
3       * Elimina un destino de la base de datos
4       *
5       * @param [type] $continent
6       * @param [type] $country
7       * @param [type] $place
8       * @return boolean True si ha podido realizar la operacion y false
9       *         ↪ en caso contrario
10      */
11     public static function delete($continent, $country, $place): bool
12     {
13         $conn = parent::conectar();
14         $query = "DELETE FROM " . TABLA_INFO . " WHERE continent = ?
15         ↪ AND country = ? AND place = ?";
16
17         try {
18             $stmt = $conn->prepare($query);
19             return $stmt->execute([$continent, $country, $place]);
20         } catch (PDOException $e) {
21             error_log('InfoTrips::delete ' . $e->getMessage());
22             return false;
23         }
24     }
25 }
```

- *infoTrips::getForCarrusel*: simplemente es una función hecha para una causa y esta es trabajar con el carrusel. Es la encargada de obtener 7 destinos aleatorios de los que haya para usarlos en el carrusel.

```
1  this.validCountries = paises.map(p => {
2      const nombreEspañol = p.translations?.spa?.common;
3      const nombreIngles = p.name?.common;
4      if (!nombreEspañol) return null;
5
6      const paisFormateado = nombreEspañol.split(' ').map(palabra =>
7          ↪ palabra.charAt(0).toUpperCase() + palabra.slice(1)).join(' ');
```

```

8         if (nombreIngles) {
9             this.countriesDictionary[paisFormateado] = nombreIngles;
10        }
11
12        return paisFormateado;
13    }).filter(Boolean).sort((a, b) => a.localeCompare(b));

```

- *infoTrips::getrelatedTrips*: se encarga de devolver una lista con, a lo sumo, 4 viajes relacionados con el *infoTrips* implícito mediante el país. Podríamos preguntarnos por qué devuelve viajes concretos con fechas. El motivo es sencillo, si quiero que sean viajes disponibles debo tratar fechas, entonces no me es ningún problema devolver los viajes y yo procesar los destinos.

```

1
2  <?php
3      /**
4       * Obtiene un array con <=4 viajes relacionados con un destino y
5       *   ↳ que sean posteriores a la fecha actual.
6       *
7       * @return array viajes relacionados con el destino, o array vacío
8       *   ↳ si no se han podido obtener los datos o no hay viajes
9       *   ↳ relacionados.
10      */
11      public function getRelatedTrips(): array
12      {
13          $conn = parent::conectar();
14          $query = "SELECT DISTINCT t.continent, t.country, t.place,
15              ↳ i.price, i.img
16                  FROM " . TABLA_VIAJES . " t
17                  INNER JOIN " . TABLA_INFO . " i
18                  ON t.continent = i.continent
19                  AND t.country = i.country
20                  AND t.place = i.place
21                  WHERE t.departureDate >= NOW()
22                  AND t.country = ?
23                  AND t.place <> ?
24                  ORDER BY t.continent, t.country, t.place
25                  LIMIT 4";
26
27          try {
28              $stmt = $conn->prepare($query);
29              $stmt->execute([$this->datos['country'],
30                  ↳ $this->datos['place']]);
31              $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
32              $relatedTrips = [];
33              foreach ($rows as $row) {
34                  $relatedTrips[] = new Trip($row);
35              }
36              return $relatedTrips;
37          } catch (PDOException $e) {
38              error_log('InfoTrips::getRelatedTrips ' .
39                  ↳ $e->getMessage());

```

```

34         return [];
35     }
36 }
37 ?>

```

- *infoTrips::tripToHtml*: se encarga de pasar el objeto implícito a una tarjeta de destino con todas sus fechas. En un inicio pertenecía a *Trip*; sin embargo, debido a que lo que estamos haciendo es asociar cada destino con todas sus fechas de viajes considero que no debe ser un método de *Trips*. Además, con este método estuve bastante rato perdido pues imprimía dos tarjetas por cada viaje cuando lo tenía implementado en *Trips*; sin embargo, cambiarlo a *infoTrips* no fue la solución, tuve que añadir *DISTINCT* a la consulta de la función *getAll*.

```

1  <?php
2  /**
3   * Convierte un viaje en una tarjeta de destino con fechas a elegir
4   *   ↳ y lo enlaza a una página u otra dependiendo de $general.
5   * $page se usa para poder gestionar las páginas en los enlaces,
6   *   ↳ simplemente se incrusta.
7   *
8   * Antes de hacer nada, toma todas las fechas asociadas a los
9   *   ↳ destinos del objeto implícito.
10  *
11  * @param [type] $general
12  * @param [type] $page
13  * @return string cadena de caracteres con el html asociado a la
14  *   ↳ carta de destino.
15  */
16  public function tripToHtml($general, $page): string
17  {
18      //Buscamos todas las fechas asociadas al viaje
19      $conn = parent::conectar();
20      $query = "SELECT departureDate, returnDate FROM " .
21      ↳ TABLA_VIAJES . "
22          WHERE continent = ? AND country = ? AND place = ? AND
23          ↳ departureDate >= NOW()
24          ORDER BY departureDate ASC";
25
26      $optionsHtml = "";
27
28      try {
29          $stmt = $conn->prepare($query);
30          $stmt->execute([
31              $this->datos['continent'],
32              $this->datos['country'],
33              $this->datos['place']
34          ]);
35          $viajesDisponibles = $stmt->fetchAll(PDO::FETCH_ASSOC);
36
37          //Construimos el select con cada resultado que hemos
38          ↳ obtenido

```

```

32         foreach ($viajesDisponibles as $viaje) {
33             // Convertimos el formato YYYY-MM-DD de la base de
34             ↪ datos a DD/MM/YYYY
35             if (!empty($viaje['departureDate']) &&
36                 ↪ !empty($viaje['returnDate'])) {
37                 $dateSalida = new
38                 ↪ DateTime($viaje['departureDate']);
39                 $fechaSalidaFormateada =
40                 ↪ $dateSalida->format('d/m/Y');
41                 $dateVuelta = new DateTime($viaje['returnDate']);
42                 $fechaVueltaFormateada =
43                 ↪ $dateVuelta->format('d/m/Y');
44             } else {
45                 $fechaSalidaFormateada = '';
46                 $fechaVueltaFormateada = '';
47             }
48
49             //El valor serán las fechas de salida y regreso
50             ↪ concretas separadas por comas
51             $value = $viaje['departureDate'] . ";" .
52             ↪ $viaje['returnDate'];
53
54             $optionsHtml .= "<option value=\"{$value}\">
55             {$fechaSalidaFormateada}-{$fechaVueltaFormateada}
56             </option>";
57         }
58     } catch (PDOException $e) {
59         error_log('Error en Trip::tripToHtml al cargar fechas: ' .
60         ↪ $e->getMessage());
61     }
62
63     //Construimos la tarjeta de viaje
64     $html = "<article class=\"viajeTipo\"><h3>" .
65     ↪ htmlspecialchars($this->datos['place']) . ", " .
66     ↪ htmlspecialchars($this->datos['country']) . "</h3>";
67
68     if ($general) {
69         $html .= "<a
70         ↪ href=\"../html/viajes_pais.php?page=1&country=" .
71         ↪ htmlspecialchars($this->datos['country']) . "\">
72         ↪ <img src=\"../imagenes/" .
73         ↪ htmlspecialchars($this->datos['img']) . "\"
74         ↪ alt=\"Imagen de " .
75         ↪ htmlspecialchars($this->datos['place']) . "\">
76         ↪ </a>
77         ↪ <form action=\"../html/viajes.php?page=" .
78         ↪ htmlspecialchars($page) . "\" method=\"post\"
79         ↪ class=\"fechaViaje\">";
80     } else {
81         $html .= "<a href=\"../html/viaje_indiv.php?continent=" .
82         ↪ htmlspecialchars($this->datos['continent']) .
83         ↪ "&country=" . htmlspecialchars($this->datos['country'])
84         ↪ . "&place=" . htmlspecialchars($this->datos['place']) .
85         ↪ "\">
86         ↪ <img src=\"../imagenes/" .
87         ↪ htmlspecialchars($this->datos['img']) . "\"
88         ↪ alt=\"Imagen de " .
89         ↪ htmlspecialchars($this->datos['place']) . "\">

```



```

66         </a>
67         <form action=\"../html/viajes_pais.php?country=\" .
        ↪ htmlspecialchars($this->datos['country']) . "\"
        ↪ method=\"post\" class=\"fechaViaje\">";
68     }
69
70     $html .= "<select name=\"fecha\" class=\"selectorFecha\">
71             <option value=\"null\">Seleccione fecha</option>
72             " . $optionsHtml . "
73             </select>
74         </form> <p class=\"precio\"><strong>\" .
        ↪ number_format($this->datos['price'], 0, ',', '.') .
        ↪ "€</strong></p>";
75
76     //Si el usuario es admin y estamos en el caso general, le
    ↪ permitimos borrar el destino
77     if (isset($_SESSION['admin']) && $_SESSION['admin'] &&
    ↪ $general) {
78         $html .= "<button class=\"deleteButton\"
79                 data-continent=\"\" .
        ↪ htmlspecialchars($this->datos['continent'])
        ↪ . "\"
80                 data-country=\"\" .
        ↪ htmlspecialchars($this->datos['country']) .
        ↪ "\"
81                 data-place=\"\" .
        ↪ htmlspecialchars($this->datos['place']) .
        ↪ "\">
82         <img src=\"../imagenes/deleteButton.png\"
        ↪ alt=\"Eliminar\">
83         </button>";
84     }
85     $html .= "</article>";
86
87     return $html;
88 }
89 }
90 ?>
91

```

- *infoTrips::getAll*: devuelve todos los destinos que tengan un viaje y que sea posterior a la fecha de hoy.

```

1  <?php
2  public static function getAll(): array
3  {
4      $conn = parent::conectar();
5      $query = "SELECT DISTINCT t.continent, t.country, t.place,
        ↪ i.price, i.img FROM " . TABLA_VIAJES . " t JOIN " .
        ↪ TABLA_INFO . " i
6          ON t.continent = i.continent AND t.country =
        ↪ i.country AND t.place = i.place WHERE
        ↪ t.departureDate >= NOW() ORDER BY t.continent,
        ↪ t.country, t.place";

```

```

7
8     try{
9         $stmt = $conn->prepare($query);
10        $stmt->execute();
11        return $stmt->fetchAll(PDO::FETCH_ASSOC);
12    } catch(PDOException $e){
13        error_log('InfoTrips::getAll: ' . $e->getMessage());
14        return [];
15    }
16 }
17 ?>

```

- *infoTrips::getAllBetween*: se encarga de dar una funcionalidad bastante completa y útil. Se usa en el buscador y en los viajes agrupados por países. Es una consulta modular donde, dependiendo de los parámetros que se le pasan crea una consulta u otra.

```

1 <?php
2 public static function getAllBetween($continent,$country,$place,
3     ↪ $departureDate, $returnDate): array
4     {
5         $conn = parent::conectar();
6
7         // Creamos la consulta base
8         $query = "SELECT DISTINCT t.continent, t.country, t.place,
9                     i.img, i.price
10                FROM " . TABLA_VIAJES . " t
11                INNER JOIN " . TABLA_INFO . " i
12                    ON t.continent = i.continent
13                    AND t.country = i.country
14                    AND t.place = i.place AND t.departureDate >= NOW()";
15
16        //Guardamos los filtros y los parametros dependiendo de lo que
17        ↪ nos den
18        $condiciones = [];
19        $parametros = [];
20
21        // Si la fecha de salida no es -1 añadimos la condicion
22        if ($departureDate !== -1 && $departureDate !== "-1") {
23            $condiciones[] = "t.departureDate >= ?";
24            $parametros[] = $departureDate;
25        }
26
27        // Si la fecha de vuelta no es -1 añadimos la condicion
28        if ($returnDate !== -1 && $returnDate !== "-1") {
29            $condiciones[] = "t.returnDate <= ?";
30            $parametros[] = $returnDate;
31        }
32
33        if ($country !== -1 && $country !== "-1") {
34            $condiciones[] = "t.country LIKE ?";
35            $parametros[] = "%" . $country . "%";
36        }
37    }
38 }

```

```

34     }
35
36     if ($continent !== -1 && $continent !== '-1' ){
37         $condiciones[] = "t.continent LIKE ?";
38         $parametros[] = "%".$continent."%";
39     }
40
41     if($place !== -1 && $place !== '-1'){
42         $condiciones[] = "t.place LIKE ?";
43         $parametros[] = "%".$place."%";
44     }
45
46     // Si tenemos alguna condicion, la añadimos a la consulta
47     if (count($condiciones) > 0) {
48         $query .= " WHERE " . implode(" AND ", $condiciones);
49     }
50
51     //Añadimos la ordenacion
52     $query .= " ORDER BY t.continent, t.country, t.place";
53
54     try {
55         $stmt = $conn->prepare($query);
56
57         //Ejecutamos la consulta
58         $stmt->execute($parametros);
59
60         return $stmt->fetchAll(PDO::FETCH_ASSOC);
61     } catch (PDOException $e) {
62         error_log("Trip::getAllBetween " . $e->getMessage());
63         return [];
64     }
65 }
66 ?>

```

- *Trip::create*: crea un viaje en la base de datos; pero, si no existe el destino que se pasa como argumento debe crearse ¹⁵.

```

1
2 <?php
3 /**
4      * Crea un viaje creando el destino si no existe o modificandolo si
5      * ↪ existe
6      *
7      * @param [type] $data
8      * @return boolean True si se crea y false si no se crea o si hay
9      * ↪ fallo
10     */
11     public static function create($data): bool
12     {
13         $conn = parent::conectar();

```

¹⁵Esta es una de las ventajas de implementar los viajes como el *JOIN* de ambas tablas.

```

12     $infoData = array(
13         "continent" => $data['continent'],
14         "country" => $data['country'],
15         "place" => $data['place'],
16         "price" => $data['price'],
17         "img" => $data['img'],
18         "lodging" => $data['lodging'],
19         "sortDesc" => $data['sortDesc'],
20         "longDesc" => $data['longDesc']
21     );
22     if (!InfoTrips::exists($data['continent'], $data['country'],
23         ↪ $data['place'])) {
24         InfoTrips::create($infoData);
25     } else {
26         InfoTrips::modify($infoData);
27     }
28
29     $query = "INSERT INTO " . TABLA_VIAJES . "(continent, country,
30     ↪ place, departureDate, returnDate) VALUES(?,?,?,?,?)";
31
32     try {
33         $stmt = $conn->prepare($query);
34         return $stmt->execute([
35             $data['continent'],
36             $data['country'],
37             $data['place'],
38             $data['departureDate'],
39             $data['returnDate']
40         ]);
41     } catch (PDOException $e) {
42         error_log('Trip::create ' . $e->getMessage());
43         return false;
44     }
45 }
46
47 ?>

```

- *Trip::getCountriesByContinents*: sólo sirve para gestionar el menú lateral de los continentes. Además, podría ser una consulta estrictamente de *infoTrips* usando lo mismo que usé en *getAll* para mostrar solo los que tengan un viaje disponible y posterior a la fecha actual. Sin embargo, para mostrar la utilidad de la tabla *Trips* he decidido dejarla aquí ¹⁶.

```

1  <?php
2  public static function getCountriesByContinents(): array
3  {
4      $conn = parent::conectar();
5      $query = "SELECT DISTINCT t.continent, t.country
6              FROM " . TABLA_VIAJES . " t

```

¹⁶No es muy útil realmente salvo para almacenar menos información redundante, simplemente no aparecerán los destinos que eno tengan viajes ahorrándome un *JOIN*.

```

7         INNER JOIN " . TABLA_INFO . " i
8             ON t.continent = i.continent
9             AND t.country = i.country
10            AND t.place = i.place
11            ORDER BY t.continent,t.country";
12
13    try {
14        $stmt = $conn->prepare($query);
15        $stmt->execute();
16        //Agrupamos por la primera columna y los valores son la
17        ↪ segunda columna
18        return $stmt->fetchAll(PDO::FETCH_GROUP |
19        ↪ PDO::FETCH_COLUMN);
20    } catch (PDOException $e) {
21        error_log("Trips::getCountriesByContinents: " .
22        ↪ $e->getMessage());
23        return [];
24    }
25 }

```

Una vez ya hemos visto cómo hemos modelado los objetos de la base de datos, podemos ya comentar el *CRUD*. Cabe recalcar que me refiero a dentro de la aplicación, como tal ya hemos creado el *CRUD* del objeto en la base de datos con estas clases en *PHP*.

8.3 Create y Update

Para crear viajes o modificarlos necesitamos ser administradores, en caso contrario no sabremos, ni siquiera, que existe esta posibilidad. El motivo de esto son las sentencias condicionantes relacionadas con ser administrador o no que aparecen en *viajes.php*, de manera que si no se cumple la condición, no se manda el código al cliente y no se visualiza el contenido. La sentencia de comprobación es la siguiente:

```

1  <?php if (isset($_SESSION['admin']) && $_SESSION['admin'] === true): ?>

```

8.3.1 Back-end

Tal y como hemos hecho en todos los formularios hasta ahora, hemos creado un manejador de formulario para el formulario de creación o modificación de viajes que se llama *formAddTrip* y que hereda de *formHandler*.

Este se encarga de obtener una serie de datos que reflejan un viaje, validar que estén todos los necesarios y realizar la modificación o creación del viaje. Y sí, digo modificación o creación porque esta clase crea el viaje y también el destino si no existe, pero si existe el destino, solo crea el viaje y modifica el destino (*infoTrips*)

con los nuevos datos. Para ello, se basa en la función *Trip::modify* que es la que se encarga de ello.

```
1  <?php
2  class FormAddTrip extends FormHandler
3  {
4      private $healthData = [];
5
6      public function validate(array $data)
7      {
8          $this->errors = [];
9
10         //Trabajamos con los datos saneados para poder modificarlos
11         $this->healthData = $data;
12
13         if (empty($data['continent'])) {
14             $this->addError('continent', 'El continente es necesario');
15         }
16         if (empty($data['country'])) {
17             $this->addError('country', 'El pais es necesario');
18         }
19         if (empty($data['place'])) {
20             $this->addError('place', 'El lugar es necesario');
21         }
22         if (!isset($data['price']) || trim($data['price']) === '' ||
23             ↪ !is_numeric($data['price'])) {
24             $this->healthData['price'] = 0.0; // 0 manejas un error de
25             ↪ validación
26         } else {
27             //Si el precio que se obtiene es un número, lo convertimos a
28             ↪ float y lo redondeamos a 2 decimales, además de evitar
29             ↪ precios negativos
30             $priceFloat = (float)$data['price'];
31
32             if ($priceFloat < 0) {
33                 $this->healthData['price'] = 0.0; // Evitamos precios
34                 ↪ negativos
35             } else {
36                 $this->healthData['price'] = round($priceFloat, 2); //Tomar
37                 ↪ dos decimales
38             }
39         }
40         if (empty($data['departureDate'])) {
41             $this->addError('departureDate', 'La fecha de salida es
42             ↪ obligatoria');
43         }
44         if (empty($data['returnDate'])) {
45             $this->addError('returnDate', 'La fecha de vuelta es
46             ↪ obligatoria');
47         }
48         if (empty($data['img'])) {
49             $this->healthData['img'] = 'default.jpg';
50         } else {
51             $img = basename($data['img']); //Se queda solo con
52             ↪ [nombre].[formato]
```

```

44
45 //Ahora debemos buscar la imagen
46 $rutaImágenes = __DIR__ . '/../imagenes/';
47 $imagenDefecto = 'default.jpg';
48
49 $rutaCompleta = $rutaImágenes . $img;
50
51 if (file_exists($rutaCompleta)) {
52     $this->healthData['img'] = $img;
53 } else {
54     $this->healthData['img'] = $imagenDefecto;
55 }
56 }
57
58 // Validamos las fechas por seguridad, aunque en la base de datos
59 ↪ también se haga
60 if (!empty($data['departureDate']) && !empty($data['returnDate'])) {
61     $fechaSalida = new DateTime($data['departureDate']);
62     $fechaVuelta = new DateTime($data['returnDate']);
63     $diferencia = $fechaSalida->diff($fechaVuelta)->days;
64
65     if ($fechaVuelta <= $fechaSalida) {
66         $this->addError('returnDate', 'La vuelta debe ser posterior
67 ↪ a la salida');
68     } elseif ($diferencia < 3) {
69         $this->addError('returnDate', 'El viaje debe durar al menos
70 ↪ 3 días');
71     }
72 }
73
74 public function process(array $data)
75 {
76     //Buscar si existe el viaje
77     $trip = Trip::getTrip($this->healthData['continent'],
78 ↪ $this->healthData['country'], $this->healthData['place'],
79 ↪ $this->healthData['departureDate'],
80 ↪ $this->healthData['returnDate']);
81
82     if ($trip == null) {
83         if (!Trip::create($this->healthData)) {
84             $this->addError('general', 'Ha habido un error al crear el
85 ↪ viaje');
86             return false;
87         }
88     } else {
89         $datos = $trip->getDatos();
90         if (!Trip::modify($datos['continent'], $datos['country'],
91 ↪ $datos['place'], $datos['departureDate'],
92 ↪ $datos['returnDate'], $this->healthData['departureDate'],
93 ↪ $this->healthData['returnDate'])) {
94             $this->addError('general', 'Ha habido un erro al modificar
95 ↪ el viaje');
96             return false;
97         }
98     }
99 }

```

```

88         }
89     }
90     return true;
91 }
92 }
93
94 ?>

```

Una cosa que es importante recalcar de esta clase es que es la encargada de comprobar si el nombre de la imagen pasada por el formulario existe y en caso de no existir colocar una imagen por defecto. Para hacer esto, usamos *basename* una función nativa de *php* que permite recortar todo aquello que no sea parte del nombre del archivo del *path* que se ha pasado en el formulario.

Realmente, el dispositivo del cliente no permite que se envíen rutas completas de archivos; no obstante, creo que aporta seguridad. Otra cosa que aporta seguridad es comprobar que las fechas pasadas son correctas y cumplen las características de la empresa como que la fecha de vuelta sea posterior a la de ida y que el viaje dure, al menos, 3 días.

Por último, toca hablar de cómo recibe los datos este manejador de formulario. Para ello, se usa una *API* que he creado en el archivo *addTrip.php* que es la encargada de recibir la petición *fetch* que explicaré en la sección de *front-end* y que, una vez recibidos los datos, los decodifica del *json* para pasarlos a un array asociativo de *php* y poder ya enviarlos al manejador de formulario. El código aparece a continuación:

```

1  <?php
2
3      require_once __DIR__ . '/../model/forms.php';
4
5      header('Content-Type: application/json');
6
7      //Obtenemos la información
8      $data = json_decode(file_get_contents("php://input"), true);
9
10     if(!$data){
11         echo json_encode([
12             "success" => false,
13             "message" => "No se han recibido datos"
14         ]);
15         exit;
16     }
17
18     //Si existen los datos, procesamos
19     $form = new FormAddTrip();
20     $status = $form->handle($data);
21
22     if(!$status){
23         echo json_encode([
24             "success"=>false,

```



```

25         "errors"=>$form->getErrors()
26     });
27     exit;
28 }
29
30 //Enviamos el exito si no hay error
31 echo json_encode([
32     "success" => true,
33     "message" => ";Viaje añadido correctamente"
34 ]);
35 exit;
36 ?>

```

8.3.2 front-end

En el cliente no hay nada distinto de lo explicado en la sección ???. Disponemos de un manejador del formulario de creación de viajes que se llama *formularioPaíses* y que hereda de *formularioBase*.

Esta clase se encarga de realizar todas las validaciones necesarias de los campos del formulario y de mostrar los errores encontrados. Los aspectos nuevos del formulario y que también aparecerán en la sección ??? son los siguientes:

- **Obtención de países válidos.** Cuando vamos a rellenar el formulario, un administrador puede no conocer muy bien la geografía planetaria e introducir países que no se corresponden con el continente que se ha elegido. Para solucionar esto, realizamos peticiones a una *API* externa que encontré con ayuda de la inteligencia artificial y que permite obtener los países del mundo según el continente que pasemos dentro de la url ¹⁷.

Para poder procesar esta información, dentro del constructor usamos una variable *validCountries* donde se almacenan los países posibles tras ejecutar *loadCountries* que es la función donde se realiza la petición *fetch*.

Dentro de esta función, una vez se realiza la petición *fetch* y se ha obtenido respuesta se ejecuta un código algo lio de leer ¹⁸ que se encarga de formatear la respuesta y cargar un diccionario con las traducciones de los países que la dispongan.

```

1  this.validCountries = paises.map(p => {
2      const nombreEspañol = p.translations?.spa?.common;
3      const nombreIngles = p.name?.common;
4      if (!nombreEspañol) return null;
5
6      const paisFormateado = nombreEspañol.split(' ').map(palabra =>
          ↪ palabra.charAt(0).toUpperCase() + palabra.slice(1)).join(' ');

```

¹⁷Si no pasamos ninguno podemos obtenerlos todos.

¹⁸Este código me ha resultado bastante complicado de hacer por mi cuenta y recurrí a la inteligencia artificial para encontrar las funciones que se comentan.

```

7
8     if (nombreIngles) {
9         this.countriesDictionary[paisFormateado] = nombreIngles;
10    }
11
12    return paisFormateado;
13 }).filter(Boolean).sort((a, b) => a.localeCompare(b));

```

El código que aparece justo arriba es el encargado de esto; con la función *map* podemos recorrer el array clave-valor de la respuesta aplicando una función. Esta función es la encargada de cargar el diccionario de traducciones de manera que si no disponemos de traducción, no se tiene en cuenta este país devolviendo *null*. Tras ejecutar *map*, se ejecuta *filter* que es la encargada de filtrar los campos según una condición pasada como argumento, en este caso, comprobar si existe el valor o es *null* ¹⁹.

Una vez ya se ha filtrado, se le aplica ordenación por orden alfabético al array para obtenerlos a todos de forma ordenada.

Ya hemos obtenido los países válidos, ahora toca construir el *datalist* asociado al campo de los países. Esto se realiza de forma sencilla como aparece a continuación:

```

1  let lista = document.getElementById("paises");
2  if (lista) {
3      lista.innerHTML = ""; //Limpiamos la lista
4      this.validCountries.forEach(nombrePais => {
5          const option = document.createElement("option");
6          option.value = nombrePais;
7          lista.appendChild(option);
8      })
9  }

```

En este código se toma el campo *datalist* que queremos rellenar y, por cada país válido, se añade una nueva entrada como opción haciendo uso de *appendChild* ²⁰.

- **Obtención de ciudades.** Esto se realiza de forma análoga al punto anterior; por tanto, lo único a comentar es que la *API* ²¹ es distinta.

Además, cabe recalcar que, como esta *API* no me permitía cargar todas las ciudades del mundo, no puede ejecutarse *loadCities* desde el principio, lo que

¹⁹Puede parecer raro, pero poniendo boolean, las cadenas de caracteres no vacías se toman como *true* y *null* como *false*.

²⁰Es una función por defecto de *javascript* que añade el argumento como hijo del objeto sobre el que se aplica. Debe ser un objeto del *DOM*.

²¹De nuevo descubierta con IA siguiendo mi petición de que buscara una *API* que admitiera peticiones *fetch* para encontrar todas las ciudades de un país.

me da paso a hablar del siguiente punto.

- **Carga de ciudades y países dinámica.** Si solo ejecutamos la función *loadCities* o la función *loadCountries* al principio, puede ser un lío para el administrador encontrar la ciudad que busca. Entonces, para que se carguen de forma dinámica, he implementado dos *EventListener* sobre los campos de continente y país. Concretamente los siguientes dos, que son análogos:
 - *setUpContinentChangeListener*. Como su nombre indica, se encarga de implementar un *EventListener* sobre el campo del continente para detectar cuándo cambia y así recargar y limpiar el campo de los países con los países referentes a ese continente. Concretamente, se implementa sobre el elemento *input* de nombre *continent* y recae en *loadCountries*.
 - *setUpCountryChangeListener*. Es la única forma de cargar las ciudades, pues como ya comenté no he encontrado la forma de cargarlas todas. Entonces, al igual que el ítem anterior, se encarga de imponer un *EventListener* que detecte los cambios en el campo de texto de *country* para poder cargar las ciudades referentes a ese país. En este caso, se usa el diccionario de países que se ha creado en la función *loadCountries* para poder trabajar correctamente con la *API* ²².

Estos dos *EventListener* se crean cuando se crea un objeto de este tipo y son los encargados de que la creación de viajes sea mucho más sencilla.

8.4 Read

La lectura de los viajes se realiza en el *back-end* haciendo uso de los archivos *viajes.php* y *viajes_pais.php* donde se muestran los países paginados por grupos de nueve en ambas páginas. Cabe destacar que, en el caso de *viajes_pais.php* se muestran los de un único país. Esta página viene enlazada desde *viajes.php* una vez que se pulsa la imagen de un destino.

Viendo que *viajes.php* y *viajes_pais.php* son idénticos salvo una consulta comentaré *viajes.php*. En esta página se muestran todos aquellos destinos que tienen viajes disponibles para fechas futuras; para obtenerlos, se usa *getAll()* donde se obtienen estos datos.

Para procesar los datos obtenidos, se trabaja con objetos de tipo *infoTrips*, de manera que, haciendo uso de *tripToHtml* se escriben en modo tarjetas de viaje. Previamente, se agrupan en grupos de máximo 9 viajes para facilitar la paginación.

Hablando de la paginación, trabajamos con un atributo *page* en la barra de navegación, es decir, en *\$_GET* que, dependiendo el valor que tome, mostrará uno de los grupos de viajes u otro. Entonces, si se pulsa la flecha derecha aumenta en uno la página permitiendo el ciclado de la misma; y si se pulsa la flecha izquierda, se disminuye en 1 la página.

²²Requiere los nombres de los países en inglés.

El movimiento de páginas se ha hecho con enlaces, lo cual ha sido una tarea sencilla. Por último, aunque ya se citó en la sección ?? se muestra, en caso de ser administrador, la posibilidad de crear los viajes mediante un formulario. El código concreto aparece a continuación:

```
1 <?php if (isset($_SESSION['admin']) && $_SESSION['admin'] === true): ?>
2 <h2>!Gestiona los viajes <?php echo $_SESSION['nickName'] ?>! </h2>
3 <form id="countryForm" action="./viajes.php" method="post">
4 <div class="field">
5 <datalist id="continentes">
6 <option value="Asia">Asia</option>
7 <option value="Africa">Africa</option>
8 <option value="Europa">Europa</option>
9 <option value="Oceania">Oceania</option>
10 <option value="America">America</option>
11 </datalist>
12 <label for="continent">Continente</label>
13 <input type="text" id="continent" name="continent"
   ↳ list="continentes">
14 </div>
15 <div class="field">
16 <label for="country">País</label>
17 <input type="text" name="country" id="country" list="paises">
18 <datalist id="paises"></datalist>
19 </div>
20 <div class="field">
21 <label for="place">Lugar</label>
22 <input type="text" name="place" id="place" list="places">
23 <datalist id="places"></datalist>
24 </div>
25 <div class="field">
26 <label for="price">Precio</label>
27 <input type="number" name="price" id="price" step="0.5" min="0">
28 </div>
29 <div class="field">
30 <label for="fechaSalida">Fecha</label>
31 <input type="date" name="departureDate" id="fechaSalida"
   ↳ min=<?php echo date('Y-m-d'); ?>>
32 </div>
33 <div class="field">
34 <label for="fechaVuelta">Fecha</label>
35 <input type="date" name="returnDate" id="fechaVuelta" min=<?php
   ↳ echo date('Y-m-d'); ?>>
36 </div>
37 <div class="field">
38 <label for="lodging">Alojamiento</label>
39 <input type="text" name="lodging" id="lodging"
   ↳ placeholder="[hotel], [albergue], [hostal]">
40 </div>
41 <div class="field">
42 <label for="sortDesc" style="display:block">Breve
   ↳ descripción:</label>
43 <textarea name="sortDesc" id="sortDesc" rows="3" columns="34"
   ↳ placeholder="Escribe una breve descripcion"></textarea>
```

```

44     </div>
45     <div class="field">
46         <label for="longDesc" style="display:block">Descripción:</label>
47         <textarea name="longDesc" id="longDesc" rows="5" columns="50"
48             ↳ placeholder="Escribe una descripcion"></textarea>
49     </div>
50     <div class="field">
51         <label for="img">Nombre de la imagen</label>
52         <input type="text" name="img" id="img"
53             ↳ placeholder="ejemplo.jpg">
54     </div>
55     <button type="submit">Done</button>
56 </form>
57
58 <!--Los scripts solo aportan funcionalidades a los administradores-->
59 <script src="../../js/main.js" type="module"></script>
60 <script src="../../js/deleteTrip.js" type="module"></script>
61
62 <?php endif; ?>

```

Queda por hablar de un archivo concreto, *viaje_indiv.php* donde, una vez dentro de *viajes_pais.php*, pulsando en la imagen, se redirige a una página de dicho viaje con toda la información y destinos relacionados con él. Esto también forma parte de la lectura de los viajes.

Para conocer completamente qué viaje mostrar, dentro de *tripToHtml* hay una distinción usando *\$general*, que cuando es *true* le otorga un enlace a *viajes_pais.php* a la imagen; pero cuando es *false*, le otorga un enlace con atributos hacia *viaje_indiv.php* donde los atributos son el continente, el país y el lugar, lo que permite trabajar correctamente para mostrar toda la información del destino.

Para obtener dicha información se usa la función *getInfoTrips* que, simplemente, busca la información relativa al destino. Tras esto solo hay que formatear una plantilla haciendo uso de *php* y obtener los destinos relacionados; para esto último, usamos la función *getRelatedTrips* y, si el resultado no es vacío, mostramos la lista de enlaces. El código concreto aparece a continuación:

```

1  <?php $relatedTrips = $infoTrip->getRelatedTrips(); ?>
2  <nav id="relatedTripsNav">
3
4      <?php if (!empty($relatedTrips)): ?>
5          <ul>
6              <?php foreach ($relatedTrips as $relatedTrip):
7                  $datos = $relatedTrip->getDatos();
8                  ?>
9                  <li>
10                     <a href="../../html/viaje_indiv.php?continent=<?php echo
11                         ↳ htmlspecialchars($datos['continent']);
12                         ↳ ?>&country=<?php echo
13                         ↳ htmlspecialchars($datos['country']); ?>&place=<?php
14                         ↳ echo htmlspecialchars($datos['place']); ?>">

```

```

11         <?php echo htmlspecialchars($datos['country'] . ", "
           ↪ . $datos['place']); ?>
12     </a>
13 </li>
14 <?php endforeach; ?>
15 </ul>
16 <?php else: ?>
17     <p>Lo sentimos, no hay viajes relacionados disponibles en este
       ↪ momento.</p>
18 <?php endif; ?>

```

8.5 Delete

8.5.1 Back-end

El modelado de esta funcionalidad en el *back-end* es simplemente una *API* que recibe el destino y las fechas, tanto de salida como de vuelta, del viaje a borrar.

Una vez procesa esto, intenta borrar el viaje en la tabla *Trips* para que desaparezca de la vista. En caso de conseguirlo devuelve éxito y, si no lo consigue, devuelve que ha habido un problema. El código aparece a continuación:

```

1 <?php
2     require_once __DIR__.'../../model/trips.php';
3
4     header('Content-Type: application/json');
5
6     //Obtenemos los datos por el canal de input
7     $data = json_decode(file_get_contents("php://input"),true);
8
9     if(!$data){
10         echo json_encode([
11             "success"=> false,
12             "message" => "No se han recibido datos"
13         ]);
14         exit;
15     }
16
17     //Si los hemos recibido, tenemos que borrar el viaje
18     $result =
       ↪ Trip::delete($data['continent'],$data['country'],$data['place']
19     , $data['departureDate'],$data['returnDate']);
20
21     echo json_encode([
22         "success"=>$result,
23         "message"=>($result)? "Se ha borrado correctamente" : "Ha habido un
       ↪ problema en el borrado"
24     ]);
25     exit;
26 ?>

```

8.5.2 Front-end

Para el *front-end*, nos hemos ayudado de un botón que se crea en el *tripToHtml* que almacena los datos en los atributos de *dataset* ²³. El código del *front-end* es el siguiente:

```
1  //////////////////////////////////////////////////
2  // EVENT LISTENER PARA ADMIN
3  //////////////////////////////////////////////////
4
5  document.addEventListener("DOMContentLoaded",()=>{
6      const buttons = document.querySelectorAll('.deleteButton');
7
8      buttons.forEach(button =>{
9          button.addEventListener('click',async (e)=>{
10             e.preventDefault();
11             //Obtenemos la tarjeta y las fechas necesarias
12             const card = button.closest('.viajeTipo');
13             const dateSelector = card.querySelector('.selectorFecha');
14             const valueDate = dateSelector.value;
15
16             if(valueDate===null || valueDate.trim()===null ||
17             ↪ valueDate.trim()==="){
18                 alert('Por favor, seleccione una fecha');
19                 return;
20             }
21
22             const [departureDate,returnDate] = valueDate.split(';');
23             //Obtenemos la info con los data-*
24             const continent = button.dataset.continent;
25             const country = button.dataset.country;
26             const place = button.dataset.place;
27             //Pedimos confirmacion
28             if(!confirm(`¿Seguro que quieres eliminar viaje a ${place} del
29             ↪ ${departureDate} ?`)){
30                 return;
31             }
32
33             //Hacemos la peticion fetch a la api;
34             const endpoint = "../php/api/deleteDestiny.php";
35             const response = await fetch(endpoint,{
36                 method:"POST",
37                 headers:{"Content-Type":"application/json"},
38                 body:JSON.stringify({
39                     continent:continent,
40                     country:country,
41                     place:place,
42                     departureDate:departureDate,
43                     returnDate:returnDate
44                 })
45             });
46
47             if(!response.ok){
```

²³Ya nos fueron útiles en la práctica 1 para las ofertas y ahora para almacenar los destinos.

```

46         throw new Error(`Error en la red: ${response.status}`);
47     }
48
49     //Si tenemos respuesta la obtenemos
50     const json = await response.json();
51
52     if(json.success){
53         //Lo único que debemos hacer es recargar la página para
54         ↪ notar el cambio
55         alert(json.message);
56         location.reload();
57     }
58     else{
59         alert(json.message);
60         window.location.href="../index.php";
61     }
62 }
63 });
64 })

```

En ellos, almacenamos el continente, el país y el lugar justo cuando se escribe el objeto implícito en *HTML*. Entonces, con un *EventListener* colocado en cada botón, controlamos la situación de la siguiente forma:

1. Una vez se pulsa el botón de borrar, se obtienen los datos si se puede. El más conflictivo es la fecha. Se actúa de la siguiente manera:
 - Si no se ha seleccionado una pareja de fechas en el selector de fecha, se muestra que no se ha seleccionado ninguna fecha con una alerta y se aborta la operación.
 - Si se ha seleccionado la pareja de fechas, se toma el valor que contiene. Concretamente, contiene el valor, tanto de salida como de vuelta y formateando esta cadena obtenemos las fechas.
2. Mostramos una alerta para confirmar que se quiere borrar el viaje concreto, mostrando el lugar y la fecha de salida.
 - Si confirma se intenta el borrado, mantando los datos con un *fetch* a la *API* que comenté en la sección anterior.
 - Si se deniega la solicitud, no se hace nada.

9 Buscador de viajes

Ya hemos explicado cómo manejar los viajes tanto en el *front-end* como en el *back-end*. Ahora toca entender cómo funciona el buscador de viajes y la página de viajes buscados.

9.1 Back-end

En el servidor hay poco que explicar, simplemente trabajamos con un formulario cuyos campos se cargan en el *front-end* para facilitar al usuario introducir correctamente los campos.

Una vez se permite pulsar el botón de búsqueda, se ejecuta la propia acción del formulario que consiste en mandar los valores por el método **GET** al archivo *viajes_buscados.php*. En él, se procesa la entrada y se buscan todos aquellos viajes que cumplen las condiciones mandadas a través del formulario.

Concretamente, se usa el método *getAllBetween* para obtener todos los viajes que comentábamos. Por último, se gestiona la paginación de los mismos de forma similar a como hacíamos con los viajes; lo único a tener en cuenta es que hay que modificar la página para gestionarla nosotros.

Cabe aclarar qué es lo que hace la función *unset* y simplemente elimina la variable *page* del array de parámetros para poder gestionarla nosotros de forma manual, es decir, de la misma forma que hacíamos en los viajes.

9.2 Front-end

Aquí hay algo más de trabajo pues, tal y como hemos hecho con todos los formularios, hemos creado una clase que será el manejador del formulario en el equipo del cliente. Concretamente, el código aparece a continuación:

```
1  <?php
2      /**
3       * Convierte un viaje en una tarjeta de destino con fechas a elegir y lo
4       * ↳ enlaza a una página u otra dependiendo de $general.
5       * $page se usa para poder gestionar las páginas en los enlaces,
6       * ↳ simplemente se incrusta.
7       *
8       * Antes de hacer nada, toma todas las fechas asociadas a los destinos
9       * ↳ del objeto implícito.
10      *
11      * @param [type] $general
12      * @param [type] $page
13      * @return string cadena de caracteres con el html asociado a la carta
14      * ↳ de destino.
15      */
16      public function tripToHtml($general, $page): string
17      {
18          //Buscamos todas las fechas asociadas al viaje
19          $conn = parent::conectar();
20          $query = "SELECT departureDate, returnDate FROM " . TABLA_VIAJES . "
21                  WHERE continent = ? AND country = ? AND place = ? AND
22                  ↳ departureDate >= NOW()
23                  ORDER BY departureDate ASC";
24
25          $optionsHtml = "";
```

```

22     try {
23         $stmt = $conn->prepare($query);
24         $stmt->execute([
25             $this->datos['continent'],
26             $this->datos['country'],
27             $this->datos['place']
28         ]);
29         $viajesDisponibles = $stmt->fetchAll(PDO::FETCH_ASSOC);
30
31         //Construimos el select con cada resultado que hemos obtenido
32         foreach ($viajesDisponibles as $viaje) {
33             // Convertimos el formato YYYY-MM-DD de la base de datos a
34             ↪ DD/MM/YYYY
35             if (!empty($viaje['departureDate']) &&
36                 ↪ !empty($viaje['returnDate'])) {
37                 $dateSalida = new DateTime($viaje['departureDate']);
38                 $fechaSalidaFormateada = $dateSalida->format('d/m/Y');
39                 $dateVuelta = new DateTime($viaje['returnDate']);
40                 $fechaVueltaFormateada = $dateVuelta->format('d/m/Y');
41             } else {
42                 $fechaSalidaFormateada = '';
43                 $fechaVueltaFormateada = '';
44             }
45
46             //El valor serán las fechas de salida y regreso concretas
47             ↪ separadas por comas
48             $value = $viaje['departureDate'] . ";" .
49                 ↪ $viaje['returnDate'];
50
51             $optionsHtml .= "<option
52                 ↪ value=\"{$value}\">{$fechaSalidaFormateada}-{$fechaVueltaFormateada}</option>";
53         }
54     } catch (PDOException $e) {
55         error_log('Error en Trip::tripToHtml al cargar fechas: ' .
56             ↪ $e->getMessage());
57     }
58
59     //Construimos la tarjeta de viaje
60     $html = "<article class=\"viajeTipo\"><h3>" .
61         ↪ htmlspecialchars($this->datos['place']) . ", " .
62         ↪ htmlspecialchars($this->datos['country']) . "</h3>";
63
64     if ($general) {
65         $html .= "<a href=\"../html/viajes_pais.php?page=1&country=" .
66             ↪ htmlspecialchars($this->datos['country']) . "\">
67             <img src=\"../imagenes/" .
68                 ↪ htmlspecialchars($this->datos['img']) . "\"
69                 ↪ alt=\"Imagen de " .
70                 ↪ htmlspecialchars($this->datos['place']) . "\">
71             </a>
72             <form action=\"../html/viajes.php?page=" .
73                 ↪ htmlspecialchars($page) . "\" method=\"post\"
74                 ↪ class=\"fechaViaje\">";
75     } else {
76         $html .= "<a href=\"../html/viaje_indiv.php?continent=" .
77             ↪ htmlspecialchars($this->datos['continent']) . "&country=" .
78             ↪ htmlspecialchars($this->datos['country']) . "&place=" .
79             ↪ htmlspecialchars($this->datos['place']) . "\">

```

```

63         <img src=\"../imagenes/\" .
        ↪ htmlspecialchars($this->datos['img']) . "\""
        ↪ alt=\"Imagen de \" .
        ↪ htmlspecialchars($this->datos['place']) . "\">
64     </a>
65     <form action=\"../html/viajes_pais.php?country=\" .
        ↪ htmlspecialchars($this->datos['country']) . "\""
        ↪ method=\"post\" class=\"fechaViaje\">;
66 }
67
68 $html .= "<select name=\"fecha\" class=\"selectorFecha\">
69         <option value=\"null\">Selecione fecha</option>
70         \" . $optionsHtml . \"
71     </select>
72 </form> <p class=\"precio\"><strong>\" .
        ↪ number_format($this->datos['price'], 0, ',', '.') .
        ↪ "&lt;/strong></p>";
73
74 //Si el usuario es admin y estamos en el caso general, le permitimos
        ↪ borrar el destino
75 if (isset($_SESSION['admin']) && $_SESSION['admin'] && $general) {
76     $html .= "<button class=\"deleteButton\"
77             data-continent=\"\" .
        ↪ htmlspecialchars($this->datos['continent']) .
        ↪ "\""
78             data-country=\"\" .
        ↪ htmlspecialchars($this->datos['country']) . "\""
79             data-place=\"\" .
        ↪ htmlspecialchars($this->datos['place']) . "\">
80         <img src=\"../imagenes/deleteButton.png\"
        ↪ alt=\"Eliminar\">
81         </button>";
82     }
83     $html .= "</article>";
84
85     return $html;
86 }
87 }
88 ?>
89

```

En él, podemos ver los siguientes aspectos:

- Una vez instanciamos un elemento de este tipo, estamos creando con él un array de países válidos (recogen los países que se pueden introducir en el campo *country* y depende del campo *continent*), un diccionario de países para poder hacer correctamente la comunicación con la *API* que gestiona las ciudades de cada país al trabajar con los países en inglés y llamamos a varias funciones; estas son:
 - *loadCountries*: al igual que pasaba en la sección ??, es la encargada de crear el *datalist* vinculado con el campo de países. Para ello, se comunica

con una *API* externa mediante una petición *fetch*; además, esta petición se realiza de una manera u otra dependiendo del contenido del continente.

- *setUpContinentChangeListener*: tal y como dice su nombre, se encarga de crear un *EventListener* que capte los cambios en el campo del continente. De manera que, si hay algún cambio, se resetea el campo del país y se realiza de nuevo la petición *fetch* asociada a los países.
- *setUpCountryChangeListener*: al igual que la anterior, coloca un *EventListener* que capte los cambios, pero esta vez en el campo de los países. Una única diferencia con la anterior es que recae en *loadCities* que es la análoga a *loadCountries* pero para las ciudades de un país.

El funcionamiento básico está explicado pero queda por comentar algunas sentencias que pueden ser raras; si miramos *loadCountries*, una vez ya se ha recibido respuesta a la petición *fetch* aparecen unas líneas de código que aparecen a continuación:

```
1  this.validCountries = paises.map(p => {
2      const nombreEspañol = p.translations?.spa?.common;
3      const nombreIngles = p.name?.common;
4      if (!nombreEspañol) return null;
5
6      const paisFormateado = nombreEspañol.split(' ').map(palabra =>
7          ↪ palabra.charAt(0).toUpperCase() + palabra.slice(1)).join(' ');
8
9      if (nombreIngles) {
10         this.countriesDictionary[paisFormateado] = nombreIngles;
11     }
12
13     return paisFormateado;
14 }).filter(Boolean).sort((a, b) => a.localeCompare(b));
```

Aquí estamos usando la propiedad *map()* de forma que podemos ejecutar una función sobre cada elemento mapeado de la respuesta; el resultado de esta función será lo que se almacene en *validCountries*.

Entonces, por cada país que haya en la respuesta vamos a tratar de traducirlo al español para poder mostrarlo correctamente a los usuarios, vamos a tomar el valor en inglés y vamos a contruir la entrada del diccionario de traducción correspondiente a este país.

Una vez disponemos del nombre en español (en caso de que se pueda, si no se obvia), lo que hacemos es formatear este nombre para que todas las palabras aparezcan con la primera letra en mayúscula y por último une las palabras.

Después, como devolvemos *null* cuando no se puede hacer la traducción, filtramos eliminando aquellos valores que no sean válidos y los ordenamos en orden alfabético según la función *localCompare* que consigue el orden habitual.

Ni que decir tiene que atribuirme este código a mí sería demasiado egoísta. Esta

parte del código se ha realizado con inteligencia artificial; concretamente he usado la versión de *Gemini* que proporciona la Universidad de Granada. No obstante, puede verse que he tratado de comprenderla hasta el último punto si así me es posible.

Tras todo ese formateo simplemente construimos el *datalist* asociado a los países. Además, cabe recalcar que el funcionamiento de *loadCities* es análogo o incluso más sencillo que el que aparece aquí; por tanto, puedo decir que lo he construido por mí mismo basándome en esta función que acabo de comentar.

10 Carrusel de viajes

10.1 Back-end

Poco hay que contar de esta parte de la aplicación web puesto que, tal y como se especifica en el guión, se debe realizar en *javascript*. Lo único a comentar es que, para poder trabajar con los viajes en el *front-end*, he creado al *API getCarrusel* que se encarga de devolver 7 viajes cualesquiera de la base de datos. El código de la *API* es el siguiente:

```
1 <?php
2     require_once __DIR__."/../model/trips.php";
3
4     header('Content-Type: application/json');
5
6     $trips = InfoTrips::getForCarrusel();
7
8     if(empty($trips)){
9         echo json_encode([
10             "success"=>false,
11             "message" => "Ups! Parece que no hay viajes ;)"
12         ]);
13         exit;
14     }
15     else{
16         //Si no ponemos & no se modifica el campo, en php se toman copias en
17         ↪ los foreach
18         foreach($trips as &$trip){
19             if(empty($trip['sortDesc'])){
20                 $trip['sortDesc'] = ";Ven a descubrir " . $trip['place'] . "
21                 ↪ con Azimut Viajes! Una experiencia inolvidable.";
22             }
23         }
24
25         echo json_encode([
26             "success"=>true,
27             "data"=>$trips
28         ]);
29         exit;
30     }
31 }
```

Lo único relevante a explicar es que, la animación de pasar de página de forma bonita, es decir, cuando desaparecen y aparecen las imágenes se ha realizado con ayuda de *CSS* y la herramienta *@keyframes*. Concretamente, he creado dos reglas para ello; la primera de ellas se llama *encendido* y refleja cuándo debe aparecer una imagen nueva, y la segunda de ellas se llama *apagado* y refleja cuándo debe desaparecer una imagen.

En definitiva, la idea es añadir o quitar dichas clases dependiendo del momento que toque para poder activar las animaciones. Una cosa nueva es el atributo *forwards* que se encarga de mantener el final de la animación hasta una futura activación.

```

1  main section#introSection article div.carrusel div.infoWrapper.apagado {
2      animation: desaparecer 1s ease-out forwards;
3  }
4
5  main section#introSection article div.carrusel div.infoWrapper.encendido {
6      animation: aparecer 1s ease-out forwards;
7  }
8
9  @keyframes aparecer {
10     from {
11         opacity: 0;
12     }
13
14     to {
15         opacity: 1;
16     }
17 }
18
19 @keyframes desaparecer {
20     from {
21         opacity: 1;
22     }
23
24     to {
25         opacity: 0;
26     }
27 }
28

```

10.2 Front-end

En esta parte se encuentra el grueso del carrusel de viajes. El funcionamiento es sencillo, dentro del *script* principal de *javascript* hay una regla para intentar localizar el contenedor donde se incrustará el carrusel; si se encuentra, se crea un nuevo objeto carrusel. El código es el siguiente:

```

1  if(document.getElementById("carrusel")){
2      new Carrusel("carrusel");
3  }

```

Este objeto carrusel se encuentra en el archivo *carrusel.js* cuyo código aparece a continuación.

```

1  //////////////////////////////////////////////////
2  // CARRUSEL DE VIAJES
3  //////////////////////////////////////////////////
4
5  export class Carrusel {
6
7      /**
8       * Carrusel que muestra los viajes destacados en la página principal
9       *
10      * @param {string} idContenedor contenedor que contiene el carrusel
11      */
12     constructor(idContenedor) {
13         this.container = document.getElementById(idContenedor);
14         this.endpoint = './php/api/getCarrusel.php';
15         this.trips = [];
16         this.currentIndex = 0;
17
18         if (this.container) {
19             this.initCarrusel();
20             this.initAutoPlay();
21         }
22     }
23
24     /**
25      * Carga los viajes destacados desde la API y los muestra en el
26      * ↪ carrusel. Si hay un error, muestra un mensaje de error en el
27      * ↪ contenedor.
28      */
29     async initCarrusel() {
30         try {
31             const response = await fetch(this.endpoint, {
32                 method: 'GET',
33                 headers: { 'Content-Type': 'application/json' }
34             });
35             if (!response.ok) {
36                 throw new Error("Ha habido un fallo con la api del
37                 ↪ carrusel");
38             }
39
40             const json = await response.json();
41
42             if (json.success && json.data.length > 0) {
43                 this.trips = json.data;

```

```

42
43         //Construimos el DOM
44         this.buildDOM();
45
46         //Actualizamos la vista
47         this.updateView();
48
49     } else {
50         this.container.innerHTML = "<p>" + json.message + "</p>";
51     }
52 } catch (error) {
53     console.error("Error al cargar el carrusel:", error);
54     this.container.innerHTML = "<p>Error de conexión al cargar los
55     ↪ destinos.</p>"
56 }
57
58 /**
59  * Construye el DOM del carrusel, añadiendo los elementos necesarios
60  ↪ para mostrar la información de los viajes y las flechas de
61  ↪ navegación. También añade los event listeners a las flechas para
62  ↪ navegar entre los viajes.
63  */
64 buildDOM() {
65     this.container.innerHTML = `
66         <div id="infoWrapper" class="infoWrapper">
67             <h3 id="carruselNames">Buscando viajes...</h3>
68             <img id="carruselImg" src="" alt="imagen carrusel">
69             <p id="infoCarrusel"></p>
70         </div>
71
72         <nav class="flechasGrupos">
73             <a href="#" id="flechaIzq">&#10094;</a>
74             <a href="#" id="flechaDcha">&#10095;</a>
75         </nav>
76     `;
77
78     //Obtenemos los elementos creados
79     this.wrapper = document.getElementById("infoWrapper");
80     this.img = document.getElementById("carruselImg");
81     this.title = document.getElementById("carruselNames");
82     this.info = document.getElementById("infoCarrusel");
83
84     const btnIzq = document.getElementById("flechaIzq");
85     const btnDcha = document.getElementById("flechaDcha");
86
87     btnIzq.addEventListener("click", (e) => {
88         e.preventDefault();
89         this.anterior();
90     });
91
92     btnDcha.addEventListener("click", (e) => {
93         e.preventDefault();
94         this.siguiente();
95     });

```



```

93     }
94
95     /**
96     * Muestra el siguiente viaje en el carrusel. Actualiza el índice actual
97     ↪ y la vista, y reinicia el temporizador de autoplay para que no se
98     ↪ pisen los temporizadores.
99     */
100     siguiente() {
101         //Actualizamos el índice
102         this.currentIndex = (this.currentIndex + 1) % this.trips.length;
103         this.updateView();
104         this.initAutoPlay();
105     }
106
107     /**
108     * Muestra el viaje anterior en el carrusel. Actualiza el índice actual
109     ↪ y la vista, y reinicia el temporizador de autoplay para que no se
110     ↪ pisen los temporizadores.
111     */
112     anterior() {
113         this.currentIndex = (this.currentIndex - 1 + this.trips.length) %
114         ↪ this.trips.length;
115         this.updateView();
116         this.initAutoPlay();
117     }
118
119     /**
120     * Actualiza la vista del carrusel con la información del viaje actual.
121     ↪ Cambia la imagen, el título y la descripción del viaje, y añade una
122     ↪ animación de transición para que el cambio sea más suave. Si no hay
123     ↪ un viaje actual o el wrapper no está definido, no hace nada.
124     */
125     updateView() {
126         const actualTrip = this.trips[this.currentIndex];
127         if (actualTrip && this.wrapper) {
128
129             this.wrapper.classList.remove("encendido");
130             this.wrapper.classList.add("apagado");
131             setTimeout(() => {
132                 this.img.src = `./imagenes/${actualTrip.img}`;
133                 this.title.innerHTML = actualTrip.place;
134                 this.info.innerHTML = actualTrip.sortDesc;
135
136                 this.img.onload = () => {
137                     this.wrapper.classList.remove("apagado");
138                     this.wrapper.classList.add("encendido");
139                 };
140
141                 //Por si no salta el onload
142                 if (this.img.complete) {
143                     this.wrapper.classList.remove("apagado");
144                     this.wrapper.classList.add("encendido");
145                 }
146             }, 1000);
147         }
148     }

```

```

140     }
141
142   }
143
144   /**
145    * Inicia el autoplay del carrusel, haciendo que se muestre el siguiente
    ↪ viaje cada 4 segundos. Si ya había un temporizador funcionando, lo
    ↪ borra para que no se pisen los temporizadores.
146   */
147   initAutoPlay() {
148     // Si ya había un temporizador funcionando, lo borramos para que no
    ↪ se pisen
149     if (this.timer) {
150       clearInterval(this.timer);
151     }
152     // Le decimos que ejecute "this.siguiente()" cada 4 segundos (4000
    ↪ ms)
153     this.timer = setInterval(() => {
154       this.siguiente();
155     }, 4000);
156   }
157
158
159 }

```

Funciona de la siguiente forma:

1. Una vez instanciado, localiza el contenedor del carrusel e inicializa los campos más importantes como son los viajes que se verán o el índice actual del viaje que se está mostrando ²⁴.

Además, si se ha encontrado el contenedor del carrusel, se inicia el carrusel y se inicia la animación automática de pasar los viajes.

2. Cuando se inicia el carrusel, se piden los viajes a la *API* antes comentada a través de *fetch* y, si se reciben correctamente y hay alguno, se crea el contenido del *DOM* asociado al carrusel mediante la función *buildDOM*. Tras eso, se actualiza la vista del carrusel con la función *updateView* que se encarga de modificar el viaje según el índice actual. Además, esta función es la encargada de la animación automática. Concretamente, como podemos ver, se encarga de refrescar el objeto quitando la clase encendido para apagarlo y comenzar con el timeout de la animación.

Entonces, en un segundo (1000 ms)²⁵ el carrusel debe hacer toda la animación de aparecer y desaparecer. Entonces, cuando ya ha desaparecido y ha cambiado la imagen, vuelve a aparecer. Por último, a modo de seguridad, si ya está la

²⁴Este dato es muy importante, pues a la hora de dar funcionalidad a las flechas del carrusel es imprescindible.

²⁵Esta cantidad debe coincidir con cada uno de las animaciones que se usan en el *CSS* apra que se vea fluido.

imagen, solo se realiza la animación.

El proceso básico es ese; sin embargo, parece que hay más cosas que trabajan por debajo para hacer todo funcionar, como son los botones de desplazamiento, los cuales se inicializan en la función *buildDOM* y no he comentado. Entonces, una vez se localizan se les asocia un *EventListener* cuando se les hace "click" para que, dependiendo del botón vayan a izquierda o derecha.

Dentro de las funciones *siguiente* y *anterior* se refleja este comportamiento cambiando el índice del viaje a mostrar, actualizando la vista y reiniciando la animación automática.

La pregunta puede ser **¿por qué reinicias la animación?** La respuesta es sencilla, si no hiciera esto, la animación no se ejecutaría de forma fluida y dejaría de estar sincronizada correctamente. Podría verse que cambia la imagen y después aparece y desaparece o que se ejecuta la animación de forma desacompañada.

Otra cosa a comentar es **¿qué hace *initAutoPlay()***? Es algo sencillo, simplemente borra el temporizador que había para la animación automática y crea otro nuevo de 4 segundos diciéndole que, cada cuatro segundos pase a la siguiente imagen.

11 Uso de IA

Volvemos a hablar sobre un tema controversial, y esta vez no puedo asegurar tanta autoría como sí hice en la práctica anterior. Lamentablemente, me he visto un poco perdido al iniciar la práctica de manera que, en algunos aspectos he necesitado guiarme de la inteligencia artificial.

Algunos de los aspectos son claros y notorios; además, creo que la mayoría los he ido comentando a lo largo de la memoria, que me ha resultado complicado escribir por no ser un código declarativo. Entonces, citémoslos:

- La primera petición *fetch*; es una herramienta nueva y no entendía muy bien cuál era el funcionamiento y en qué partes debía poner el dichoso *await*. A día de hoy creo que soy capaz de hacer peticiones de este estilo, con más o menos tiempo necesitado.
- El formateo de la respuesta de la *API RestCountries* para poder trabajar con los *datalists* en el formulario de búsqueda; para mí fue un dolor de muelas trabajar con esos formatos. Tanto fue así que recurrí a la IA, quien me propuso esta mejora.
- El primer *EventListener*, que recuerdo perfectamente que fue el botón de registrarse en el alta de usuarios. Lo usé porque no entendía bien cómo se creaba el evento y se recogía. Hasta que, gracias a la IA entendí que el evento funciona directamente una vez se crea, no necesita recogerse.
- El uso del dichoso *setTimeout* en el carrusel de viajes. Tuve que usarla para entender por qué no me aparecía sincronizada la animación de *CSS* y el cambio

de imagen en *javascript*. Esta herramienta me permitió entender que debe durar exactamente lo mismo que cada una de las animaciones que uso.

- Además, al igual que en la práctica anterior, he usado esta herramienta para descubrir nuevos aspectos o herramientas que desconocía para después investigarlas un poco en las documentaciones de la bibliografía.

Por último, cabe destacar que el único agente que he usado ha sido *Gemini* en la versión proporcionada por la universidad.

12 Bibliografía

Referencias

- [1] MoureDev. *Curso COMPLETO de JavaScript DESDE CERO para Principiantes*. MoireDev. URL: <https://www.youtube.com/watch?v=1glVfFxj8a4>.
- [2] Mozilla Resources. *JavaScript Guide*. MDN Web Docs. URL: <https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>.
- [3] Mozilla Resources. *KeyFrames CSS*. MDN Web Docs. URL: <https://developer.mozilla.org/es/docs/Web/CSS/Reference/At-rules/@keyframes>.
- [4] Mozilla Resources. *setTimeout()*. MDN Web Docs. URL: <https://developer.mozilla.org/es/docs/Web/API/Window/setTimeout>.
- [5] Mozilla Resources. *Using fetch*. MDN Web Docs. URL: https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch.
- [6] PHP. *PHP Array Tutorial*. PHP. URL: <https://www.php.net/manual/es/language.types.array.php>.
- [7] W3Schools. *PHP Tutorial*. W3Schools. URL: <https://www.w3schools.com/php/default.asp>.